Replenishing perishables on shared delivery routes: Stock-age-based inventory policies, stochastic lookahead and decision-focused learning

Abstract

For a periodic-review perishable inventory-routing system with contextual forecasts, FIFO issuing, lost sales and shelf capacity, we propose decision-focused learning (DFL) at two levels: on top of a deliver-up-to-level inventory rule, learning context- and stock-age-dependent order-up-to levels from the realised cost of whole episodes (UL-DFL), and on top of a rolling-horizon inventory-routing model, learning its planning targets (LA-DFL). Twelve policies are compared in closed loop with identical information, from base-stock rules and the rule of Crama et al. (2018) to deterministic, robust and scenario-based lookahead (Jia et al., 2025) solved to a proven 0.5% gap. We show that the inventory recourse admits FIFO-optimal solutions once early disposal is allowed and that DFL stationary points reduce to the perishable newsvendor fractile. Across thirteen settings, including three network layouts and observed fresh-food demand, UL-DFL is cheaper than its rule in every setting, significantly in eleven (0.5–1.2%), and 2.9–10.3% cheaper than the best integrated policy unless routing is expensive. In the base case it serves the same fill rate with a fifth less waste, which no parameterisation of the rule achieves; a rule tuned in hindsight matches its cost but wastes 41% more. It learns to build stock before weekly demand peaks. The same learner without the rule's structure is never cheaper than UL-DFL. Only with expensive routing is scenario-based lookahead best. LA-DFL improves the deterministic lookahead by up to 5% by learning that only the implemented day needs a high safety quantile, a gain that a day-specific quantile level reproduces.

Keywords: Perishable inventory; Inventory routing; Waste; Lost sales; Base-stock policies; Decision-focused learning.

1 Introduction

Fresh food is replenished daily, its stock ages every night, and whatever is not sold within its shelf life is thrown away. Retailers and their distributors therefore face the classical perishable inventory trade-off between waste and lost sales [4, 37], but with two complications that single-store inventory models do not capture. First, deliveries to different stores share a small fleet: a store that is not on today's route cannot be replenished today, and a vehicle that is full cannot top up the next store. Second, retailers now have rich contextual information—promotion calendars, weather forecasts, recent sales—that machine-learning forecasters turn into store-day demand predictions. The operational question is how such forecasts should be translated into replenishment and delivery decisions when stock ages, shelf space is limited and vehicles are shared.

Two literatures answer different halves of this question. Perishable inventory control offers simple, robust ordering rules—base-stock policies [10], and rules that anticipate the waste of the current stock [9, 23]—but treats each store in isolation. Inventory routing integrates replenishment and routing, with exact methods for deterministic perishable demand [1, 11], decomposition and order-up-to rules for stochastic perishable demand [14], robust formulations [24, 41] and, recently, scenario-based predict-then-optimise policies that use machine-learning quantile forecasts [32]. In parallel, decision-focused learning (DFL) trains forecasters on the cost of the decisions they induce rather than on forecast accuracy [17, 35]. Whether DFL adds anything once the strong inventory rules and stochastic lookahead methods are on the table has not been tested for perishable inventory routing.

This paper studies a periodic-review perishable inventory-routing system with contextual demand, lost sales, FIFO issuing and shelf capacity, and compares twelve replenishment policies in closed loop, every day observing the age-structured stock and the same forecasts. The comparison includes the inventory-control rules a perishable-inventory manager would use (base-stock with and without estimated waste), the deliver-up-to-level rule of Crama et al. [14], and rolling-horizon inventory-routing models driven by mean, quantile, robust and scenario forecasts, the last being the Scenario Predict-then-Optimize method of Jia et al. [32]. We apply decision-focused learning at two levels. UL-DFL keeps the visit trigger of the deliver-up-to-level rule and learns how far each store's order-up-to level should deviate from the forecast quantile, depending on context and on the age of the store's stock; the learning signal is the realised holding, waste and lost-sales cost of whole training episodes, differentiated through the FIFO dynamics. LA-DFL keeps a deterministic rolling-horizon plan and learns its planning targets, differentiating the realised cost of the plan's deliveries through a coverage layer while an exact MILP oracle fixes the routes.

The answer is that learning pays when it is placed on the right structure. Across the settings studied, the stock-age-based rules with their orders routed optimally were cheaper than every integrated policy unless travel was expensive, and UL-DFL improved the best of them in every setting, significantly in eleven of thirteen, which made it the cheapest policy overall, or tied for cheapest, except with expensive routing. It does so mainly by serving the same fill rate with less waste than any parameterisation of the rule can; a rule tuned in hindsight on the test data matches its cost but not its waste. The gain is smallest where the rule already wastes little or where demand is too variable for the context to help, and it depends on the rule: the same learner without the rule's structure was never cheaper than UL-DFL and mostly not cheaper than the rule itself. Its corrections are interpretable: lower order-up-to levels on promotion days and after the weekly demand peak, when leftover stock meets falling demand. On top of the deterministic lookahead, decision-focused learning revealed that only the executed day needs a high safety quantile; a lookahead with day-specific quantile levels reproduces that gain without learning.

The paper makes three contributions.

1. **Decision-focused learning of perishable order-up-to levels on shared routes.** UL-

DFL learns context- and stock-age-dependent order-up-to levels for a published perishable inventory-routing rule from the realised cost of whole episodes, with model selection on the closed-loop system including exact routing. It is cheaper than the rule it starts from in every setting, significantly in eleven of thirteen, and the cheapest or joint-cheapest policy unless routing is expensive; its corrections can be read as inventory rules, and an ablation without the rule shows that the structure is what makes the learning pay. We show that the inventory recourse admits FIFO-optimal solutions once early disposal is allowed, bound the marginal value of a delivery by the lost-sales and waste penalties, and prove that stationary points of the decision-focused objective of the lookahead learner satisfy the stochastic optimality conditions of the fixed-route problem, reducing to the perishable newsvendor fractile for one-day shelf life; all results are verified numerically.

2. **An inventory-centred benchmark of replenishment policies under routing coupling.** We evaluate stock-age-based inventory rules, a published stochastic PIRP rule, and integrated deterministic, robust, scenario-based and decision-focused lookahead policies under one closed-loop simulator, with every lookahead solved by an exact MILP to a proven gap, three network layouts, 100 test episodes per setting, three training seeds, prespecified hypotheses, and a case with observed fresh-food demand. Outcomes are reported in inventory terms—cost composition, fill rate, waste rate, delivery frequency and store-level service—across shelf life, cost ratio, fleet tightness, routing cost, demand variability and issuing policy.

3. **Managerial evidence on when integration, stochastic lookahead and learning pay off.** The results show when simple stock-age-based rules suffice and when integration with routing pays, why deterministic plans over-stock and how a day-specific safety stock repairs this, which corrections of an order-up-to rule reduce waste, and how integrated plans concentrate stock-outs at individual stores (Section 6).

Section 2 reviews related work, Section 3 defines the problem and its structural properties, Section 4 describes the policies, Section 5 the experimental design, Section 6 the results, and Section 7 concludes with managerial implications and limitations.

2 Related literature

2.1 Perishable inventory control

Perishable inventory theory studies ordering when units have a finite lifetime and the age of stock matters for issuing, outdating and service [37]; Bakker et al. [4] and Janssen et al. [31] review the subsequent literature. Optimal policies depend on the full age vector and are computed by stochastic dynamic programming only for small instances, so practice relies on simple rules. Base-stock policies that raise the total inventory to a target are the default in automated retail ordering, and Bu et al. [10] show that they become asymptotically optimal when lifetime, demand volume or the penalty costs grow. Stock-age information helps when lifetimes are short. Broekmeulen and van Donselaar [9] propose to add the estimated waste of current stock to the base-stock order (EWA), Haijema [21] studies optimal ordering, issuing and disposal decisions, Haijema and Minner [22] compare stock-level dependent hybrid base-stock and constant-order rules, and Haijema and Minner [23] show in 11,177 instances that a base-stock policy with an estimated-waste correction (BSP-low-EW) performs close to the SDP optimum and better than the other practical rules. Deep reinforcement learning has been applied to perishable inventory control [8, 15, 44] and matches strong heuristics in single-location settings [19]. Data-driven newsvendor work uses features to set perishable order quantities directly [5, 29]. All of these studies consider a single stocking point or decouple stores; deliveries do not compete for vehicles and the cost of a delivery does not depend on which other stores are visited.

2.2 Perishable inventory routing

Inventory routing integrates replenishment with vehicle routing [13]. For perishables, Coelho and Laporte [11] and Alvarez et al. [1] develop exact formulations and branch-and-cut for deterministic demand with age-dependent inventory, and Soysal et al. [42] add demand uncertainty and emissions in a chance-constrained model. Crama et al. [14] study stochastic perishable inventory routing and compare an expected-value policy, the deliver-up-to-level rule UL_λ, and decomposition methods; they find that the simple rule is competitive in some settings and that more advanced methods pay off in others. He et al. [24] study a robust perishable IRP under demand uncertainty and solve robust-counterpart reformulations by Benders decomposition and an iterated local search. For non-perishable IRPs under uncertainty, Solyalı et al. [41] derive robust counterparts, Coelho et al. [12] show that rolling-horizon policies using stochastic information outperform deterministic ones, and Malicki and Minner [34] integrate dynamic safety stocks into cyclic inventory routing under non-stationary demand. Most closely related to our question, Jia et al. [32] propose Scenario Predict-then-Optimize: quantile neural forecasts generate demand scenarios for a stochastic rolling-horizon IRP, and the single-stage scenario model performs best on synthetic and industrial data; their model has backlogging and no product ageing. Greif et al. [20] learn the parameters of a prize-collecting TSP policy for dynamic IRP with machine learning. None of these works compares stock-age-based inventory rules, robust and scenario-based lookahead and decision-focused learning under one closed-loop simulator with perishability.

2.3 Contextual and decision-focused optimisation

When demand forecasts feed an optimisation model, one can estimate first and then optimise, or train the forecaster on the downstream cost [17, 39]. Decision-focused learning (DFL) differentiates through the optimisation problem, either exactly for convex problems [3, 16] or through surrogates and perturbations for combinatorial ones [6, 35, 43]; solution caching reduces the number of solver calls [36]. The theory is nuanced: under a well-specified model, estimate-then-optimise can converge faster and stochastically dominate integrated training [18, 27], so gains from DFL should be expected mainly when the downstream model is a misspecified approximation. In inventory control, Alvo et al. [2] train neural ordering policies by differentiating simulated costs through known inventory dynamics. For inventory routing, an early DFL prototype exists only as a workshop abstract without perishability [30]. Our method differentiates realised perishable inventory cost through a deterministic rolling-horizon PIRP whose routes are fixed by an exact MILP, so that learning corrects the safety stock embedded in the plan rather than the routing decisions.

2.4 Positioning

Table 1 positions the study. It is the first, to our knowledge, to evaluate decision-focused learning for perishable inventory routing against the inventory-control rules recommended in the perishable inventory literature and against robust and scenario-based lookahead methods, under identical information and an exact optimisation layer. The methodological novelty is the decision-focused target layer with its structural justification; the managerial contribution is evidence on when integration, stochastic lookahead and decision-focused learning are worth their cost.

Table 1: Positioning relative to the closest studies.

Study	Ageing	Uncertain demand	Routing	Context	Closed loop	Methods compared
Haijema and Minner [23]	✓	✓	–	–	✓	Stock-age rules vs SDP
Broekmeulen and van Donselaar [9]	✓	✓	–	–	✓	EWA vs base-stock
Coelho and Laporte [11]; Alvarez et al. [1]	✓	–	✓	–	–	Exact MILP
Crama et al. [14]	✓	✓	✓	–	✓	EV, UL_λ, decomposition
He et al. [24]	✓	✓	✓	–	–	Robust, Benders, ILS
Solyalı et al. [41]	–	✓	✓	–	–	Robust MILP
Jia et al. [32]	–	✓	✓	✓	✓	Scenario PTO vs PTO, SAA, DFL
Islam Sohan and Wasi [30]	–	✓	✓	✓	–	DFL prototype
This study	✓	✓	✓	✓	✓	Stock-age rules, UL_λ, robust, scenario PTO, DFL

3 Problem description

3.1 The periodic-review perishable inventory-routing system

A distributor replenishes a single fresh product at n retail stores $\mathcal{N} = \{1, \dots, n\}$ from a depot 0. Every morning (day t) it decides how much to deliver to each store and how to route a homogeneous fleet of K vehicles of capacity Q; each vehicle performs at most one route per day and each store is visited at most once per day. Deliveries arrive before the store opens (zero lead time), as is common for daily fresh-food replenishment. Products arrive fresh and can be sold on m consecutive days, the day of delivery included. Store i can hold at most C_i units at opening (shelf and backroom capacity).

State and stock age. Let B_{irt} denote the units at store i at the opening of day t, before the delivery, that can still be sold on r days including day t ($r = 1, \dots, m-1$). The vector $B_t = (B_{irt})$ is the age-structured inventory state observed by the planner. Units with $r = 1$ are on their last day.

Daily dynamics. Given deliveries $q_t = (q_{it})$ and realised demand D_{it}, the following happens at each store. (i) Available stock by remaining life is $S_{irt} = B_{irt}$ for $r < m$ and $S_{imt} = q_{it}$. (ii) Demand is served first-in first-out (FIFO), oldest units first. (iii) Unmet demand is lost. (iv) Unsold units with $r = 1$ expire. (v) Remaining units are carried overnight and lose one day of remaining life. With sales $w_{irt} = \min\{S_{irt}, (D_{it} - \sum_{r'<r} w_{ir't})^+\}$, lost sales are $u_{it} = D_{it} - \sum_r w_{irt}$, expired units are $e_{it} = S_{i1t} - w_{i1t}$, and the next state is $B_{i,r,t+1} = S_{i,r+1,t} - w_{i,r+1,t}$ for $r = 1, \dots, m-1$. As a robustness check we also simulate partial last-in first-out (LIFO) issuing, in which a fraction of customers picks the freshest units [21].

Costs. Day t costs

$$c_t = c^{\mathrm{r}}\,\mathrm{TSP}(\mathrm{routes}_t) + h \sum_{i,r} B_{i,r,t+1} + p_s \sum_i e_{it} + p_u \sum_i u_{it}, \tag{1}$$

where $\mathrm{TSP}(\cdot)$ is the travelled distance, h the holding cost per unit carried overnight, p_s the outdating (waste) cost per expired unit and p_u the lost-sales penalty. Throughout, $h \le p_s < p_u$: carrying a unit one more night is cheaper than discarding it, and a lost sale is the most expensive event.

Information and policies. Before deciding on day t the planner observes the state B_t, the demand history up to day $t-1$, and contextual information about the next days: calendar, announced promotions and holidays, and a weather forecast. A *policy* maps this information to deliveries and routes. The objective is to minimise the expected cost $\mathbb{E}\big[\sum_{t=1}^{T} c_t\big]$ over a horizon of T days. Delivery decisions couple days through stock age and stores through vehicle capacity and route costs; this is what distinguishes the problem from single-store perishable inventory control [23, 37] and makes it a stochastic perishable inventory-routing problem [14].

3.2 Lookahead model

All integrated policies in Section 4 solve, every day, an H-day lookahead model and implement only its first day (rolling horizon). The model takes the observed state B_t and one or several planning demand paths $\tilde{d}^s = (\tilde{d}^s_{ik})_{i\in\mathcal{N},k=0..H-1}$, $s \in \mathcal{S}$, as inputs. Its routing part uses a vehicle-assignment formulation with exact route costs. Let $\mathcal{R} = 2^{\mathcal{N}}$ be the set of store subsets and $\mathrm{TSP}(R)$ the length of an optimal depot-to-depot tour through R, precomputed by the Held–Karp recursion [25]. Binary z_{ivk} assigns store i to vehicle v on lookahead day k; continuous $y_{Rvk} \geq 0$ selects the route of vehicle v; $g_{ivk} \geq 0$ is the load for store i on vehicle v; $q_{ik} = \sum_v g_{ivk}$.

$$\min \sum_{k,v,R} c^{\mathrm{r}}\mathrm{TSP}(R)\, y_{Rvk} + \frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}}\sum_{k,i}\Big(h\sum_{r\geq 2}(S^s_{irk} - w^s_{irk} - \zeta^s_{irk}) + p_s(S^s_{i1k} - w^s_{i1k}) + p_s\sum_{r\geq 2}\zeta^s_{irk} + p_u u^s_{ik}\Big) \tag{2}$$

$$\text{s.t.}\quad \textstyle\sum_R y_{Rvk} = 1, \quad \sum_{R\ni i} y_{Rvk} = z_{ivk}, \quad \sum_v z_{ivk} \leq 1 \qquad \forall i, v, k, \tag{3}$$

$$g_{ivk} \leq \min\{Q, C_i\}\, z_{ivk}, \quad \textstyle\sum_i g_{ivk} \leq Q \qquad \forall i, v, k, \tag{4}$$

$$S^s_{imk} = q_{ik}, \quad S^s_{ir0} = B_{irt}\ (r < m) \qquad \forall i, r, k, s,$$

$$S^s_{i,r,k+1} = S^s_{i,r+1,k} - w^s_{i,r+1,k} - \zeta^s_{i,r+1,k} \qquad \forall i, r, k, s, \tag{5}$$

$$\textstyle\sum_r w^s_{irk} + u^s_{ik} = \tilde{d}^s_{ik}, \quad w^s_{irk} + \zeta^s_{irk} \leq S^s_{irk}, \quad \sum_r (S^s_{irk} - \zeta^s_{irk}) \leq C_i \qquad \forall i, r, k, s, \tag{6}$$

with all variables non-negative and z binary. Constraint (3) lets each vehicle choose one route (the empty set has zero cost) whose store set equals the stores assigned to it; (4) enforces vehicle and shelf capacity; (5) is the age-structured balance; (6) links sales, lost sales and the shelf capacity at opening. ζ is an excess-disposal slack priced at p_s; it is only needed when $|\mathcal{S}| > 1$ (in a deterministic plan stock never exceeds what was planned) and it is never used on day 0, where the state is known. With $|\mathcal{S}| = 1$ the model is a deterministic rolling-horizon PIRP; with $|\mathcal{S}| > 1$ it is the single-stage scenario model (one routing and delivery plan for all scenarios) that Jia et al. [32] found to perform best. A symmetry-breaking constraint assigns the lowest-indexed visited store to vehicle 1.

Proposition 1 (Exact routing cost)**.** *For every binary z satisfying (3), the only feasible y sets $y_{R_{vk},v,k} = 1$ for $R_{vk} = \{i : z_{ivk} = 1\}$. Hence the routing term of (2) equals the exact travelled distance of the assigned routes, and the model is an exact MILP formulation of the H-day lookahead PIRP in which only the nKH assignment variables are integer.*

The proof is in Appendix A. Proposition 1 matters computationally: with ten stores the model has $2^{10}KH$ continuous route columns but only nKH binaries, whereas the arc-based vehicle-indexed formulation used in the original submission could not be solved within its time budget. Because only day 0 of each plan is implemented, the experiments impose integrality on the day-0 assignment and relax it on later lookahead days; the relaxed routing term is then a convex lower estimate of future travel cost. With this choice HiGHS [28] proves a gap of at most 0.5% for every decision within the time limit (Section 6.8); in a pilot on four episodes, imposing integrality on all lookahead days changed episode costs by between -1.5% and $+3.3\%$ at three to five times the solution time and with time-limit stops. Day-0 routes of every implemented decision are additionally re-optimised by exact enumeration of all feasible route sets.

3.3 Structural properties

The results below connect the lookahead model with the physical FIFO system and justify the learning signal used in Section 4.5. Fix one store, a delivery plan $q = (q_0, \dots, q_{H-1})$, an initial state B and a demand path d. Let $C_{\mathrm{FIFO}}(q; d)$ be the inventory cost (holding, waste and lost sales) of the physical system above, and let $V_{\mathrm{LP}}(q; d)$ be the optimal value of the *recourse LP*: constraints (5)–(6) for one scenario with q fixed, the issuing order left free and early disposal ζ allowed.

Theorem 1 (Complete recourse and FIFO optimality)**.** *(i) For any state with $\sum_r B_{ir} \le C_i$ and any non-negative planning demand, the lookahead MILP is feasible and its optimal value is finite. (ii) For fixed deliveries, the recourse LP has an optimal solution in which every day's sales follow FIFO and demand is served whenever stock is available. (iii) $V_{\mathrm{LP}}(\cdot; d)$ is convex and piecewise linear, and $V_{\mathrm{LP}}(q; d) \le C_{\mathrm{FIFO}}(q; d)$, with equality whenever early disposal is not profitable along the FIFO trajectory.*

Part (ii) corrects a claim of the original submission, which stated that age-increasing holding costs make every optimal solution FIFO without a disposal option. That statement is false: if an expiring unit and a fresh unit are on the shelf and only one more unit will ever be sold, selling the fresh unit and letting the old one expire saves the fresh unit's holding cost. Allowing disposal at cost p_s restores the exchange argument (the younger unit can always replicate the older unit's path) and makes the recourse a convex relaxation of the physical system.

Proposition 2 (Marginal value of a delivery)**.** *(a) The dual multiplier π_k of the fresh-stock constraint $S_{imk} = q_k$ is a subgradient of V_{LP} with respect to q_k, and $-p_u \le \pi_k \le p_s$. (b) If $h \le p_s$, $C_{\mathrm{FIFO}}(\cdot; d)$ is piecewise linear and Lipschitz, and its one-sided partial derivatives satisfy*

$$-p_u \;\le\; \frac{\partial C_{\mathrm{FIFO}}}{\partial q_k} \;\le\; p_s + (H - 1 - k)\, h. \tag{7}$$

Proposition 2 states the familiar newsvendor logic in a multi-period, age-structured form: an extra delivered unit can at most prevent one lost sale, and at worst it is carried to the end of the horizon or wasted (possibly by displacing a later delivery). The asymmetry $p_u \gg p_s$ is what makes point forecasts a poor input for perishable replenishment. The bound in (b) is tight (Section 6.9).

The learned planning targets of Section 4.5 enter the lookahead through the deliveries they induce. For a fixed visit pattern, a delivery on day k covers the planned demand of the days until the next visit, limited by shelf life and horizon, net of the stock that will still be usable. Let W_k denote this coverage window.

Proposition 3 (Decision-focused stationarity)**.** *Fix the visit pattern and consider planning targets $\tilde{d}_j$ at which no capacity binds and the coverage map is differentiable. Then $\partial q_k / \partial \tilde{d}_j \in \{0, 1\}$, with value one exactly when day j lies in the coverage window W_k and is not already covered by planned stock, and for such j*

$$\frac{\partial}{\partial \tilde{d}_j}\, \mathbb{E}\big[C_{\mathrm{FIFO}}(q(\tilde{d}); D) \mid \xi\big] = \mathbb{E}\Big[\frac{\partial C_{\mathrm{FIFO}}}{\partial q_{k(j)}} \,\Big|\, \xi\Big], \tag{8}$$

where $k(j)$ is the visit covering day j. A stationary point of the decision-focused objective therefore satisfies the first-order optimality condition of the stochastic program in which only the delivery quantities of the fixed visit pattern are optimised. With a single-day shelf life, (8) vanishes exactly at the perishable newsvendor quantile, $F_{D|\xi}(\tilde{d}) = p_u/(p_u + p_s)$.

Proposition 3 clarifies what decision-focused training can and cannot do. It can correct the safety stock embedded in deterministic plans, store by store and context by context, which

is where the cost asymmetry and stock age act; it does not optimise visit days, which are left to the MILP. It also explains the relation to the estimate-then-optimise literature: when the forecasting model is well specified and decisions are optimised against the full predictive distribution, integrated training has no first-order advantage [18, 27]; a gain is possible only when the downstream model is a misspecified deterministic approximation, as all rolling-horizon PIRP policies are.

4 Replenishment policies

All policies receive the same information each day: the observed age-structured stock B_t and the same contextual forecasts. They differ only in how the forecasts and the stock are turned into deliveries and routes. Table 2 summarises the twelve policies; parameters marked as tuned are selected on validation episodes only. Decision-focused learning is applied at two levels: on top of the deliver-up-to-level rule (UL-DFL, Section 4.3), where it learns the order-up-to level, and on top of the deterministic lookahead (LA-DFL, Section 4.5), where it learns the planning targets.

4.1 Shared forecasts

For every store i, planning day t and lookahead offset $k \in \{0, \dots, H-1\}$, one LightGBM model [33] predicts the mean $\hat{\mu}_{i,t+k}$ and eleven LightGBM quantile models predict the quantiles $\hat{Q}_\tau(D_{i,t+k} \mid \xi)$ for $\tau \in \{0.05, 0.1, 0.2, \dots, 0.9, 0.95\}$ (sorted to avoid crossing). Features are store identity, offset k, calendar, announced discount and holiday, temperature forecast, and summaries of the last 1, 7 and 28 days of demand. A forecast scale is $\hat{\sigma} = (\hat{Q}_{0.9} - \hat{Q}_{0.1})/(2z_{0.9})$, and quantiles at other levels are linearly interpolated. Quantile regression of this type is the standard data-driven newsvendor input [5, 29].

4.2 Inventory-control rules with route-second delivery

These policies represent current replenishment practice: an inventory rule sets the store orders, and the orders are then routed. Deliveries are routed by an exact capacitated vehicle-routing oracle that enumerates all feasible route sets (possible because $n = 10$). If the fleet cannot carry all orders, the part exceeding today's median demand is reduced proportionally until a feasible packing exists.

BSP: periodic base-stock. Stores are served on a fixed cycle of $R \in \{1, 2\}$ days (for $R = 2$ in two alternating groups). On a delivery day, store i is raised to the order-up-to level $S_{it} = \hat{Q}_\alpha\big(\sum_{k<R} D_{i,t+k}\big)$ (normal approximation for $R > 1$). The inventory position counts all units regardless of age, as in most retail ordering systems [23].

BSP-EW: base-stock with estimated waste. Identical to BSP, except that units of the current stock that are expected to expire unsold during the cycle are removed from the inventory position: $q_{it} = (S_{it} - \sum_r B_{irt} + \widehat{EW}_{it})^+$, where $\widehat{EW}_{it}$ is computed by projecting the age profile forward under FIFO with the mean forecasts. This is the "estimated withdrawal and aging" idea of Broekmeulen and van Donselaar [9]; waste-anticipating base-stock rules of this family were found to be near-optimal among practical stock-age-dependent policies by Haijema and Minner [23].

UL$_\lambda$: deliver-up-to-level rule [14]. A store is visited when its stock does not cover the current day's demand at service level β, i.e. when $\sum_r B_{irt} < \hat{Q}_\beta(D_{it})$; it then receives enough to reach the α-quantile of the demand of the next λ days, net of stock expected to expire. This is the state-dependent "deliver-up-to-level" rule that Crama et al. [14] identify as a simple policy

with good performance in some settings; the original paper combines it with its own routing heuristic, whereas here the selected deliveries are routed exactly.

4.3 Decision-focused order-up-to levels (UL-DFL)

UL-DFL (proposed) keeps the visit trigger and the delivery logic of UL_λ and learns how far the order-up-to level of each store-day should deviate from the forecast quantile:

$$\tilde{S}_{it}(\theta) = S_{it} + \hat{s}_{it}\,\delta_\theta(\phi_{it}), \qquad q_{it} = \mathbf{1}\Big[\textstyle\sum_r B_{irt} < \hat{Q}_\beta(D_{it})\Big]\,\Big(\tilde{S}_{it} - \textstyle\sum_r B_{irt} + \widehat{EW}_{it}\Big)^+, \tag{9}$$

where $S_{it} = \hat{Q}_\alpha\big(\sum_{k<\lambda} D_{i,t+k}\big)$ is the tuned rule's level, $\hat{s}_{it}$ the forecast scale of the λ-day demand, and δ_θ a two-layer network (32 SiLU units per layer) whose output layer is initialised to zero, so that training starts exactly at the tuned UL_λ. The input ϕ_{it} contains the store's forecast features for day t (discount, promotion and holiday flags, temperature forecast, weekday, recent demand), the scaled mean, scale and 5–95% range of the forecast, and the store's stock by remaining life. A correction $\delta_\theta = 0.5$ raises the level by half a forecast standard deviation; under a normal approximation it moves the implied service level from α to $\Phi(z_\alpha + \delta_\theta)$.

The rule covers only λ days and ignores what happens to the stock afterwards, in particular that units delivered today are carried into tomorrow and expire after m days. UL-DFL learns this from the realised cost of whole episodes. On a training episode the rule is simulated for all 21 decision days with the age-structured FIFO dynamics of Section 3.1, written as a differentiable program: sales, waste and carried stock are compositions of minima and differences of the stock vector and the deliveries, shelf capacity caps each delivery, and fleet capacity scales the day's deliveries down proportionally when their total exceeds KQ. The visit trigger is held fixed (it is piecewise constant in θ). Back-propagation through time then returns the gradient of the episode's holding, waste and lost-sales cost with respect to θ. Following Proposition 2, an extra unit in a delivery is pushed up when, in hindsight, it would have prevented a lost sale on any later day, and pushed down when it would have been carried for several nights or wasted; unlike the lookahead-based learner below, these effects are traced over the whole episode rather than over the H days of a plan. Neural ordering policies trained through simulated inventory dynamics in this way have been proposed for single-location lost-sales systems by Alvo et al. [2]; we use the same principle to correct, rather than replace, a published PIRP rule.

Exact packing, routing and routing cost do not enter the gradient. They enter model selection: after every epoch the current network is run in the full closed-loop simulator on the validation episodes, and the network with the lowest validation cost, which may be the tuned rule itself, is kept. We train three networks from different random seeds and keep the one with the lowest validation cost. Once trained, UL-DFL needs one forward pass per store and day, so it is as fast online as UL_λ.

NN-DFL: the same learner without the rule. To separate the contribution of the rule's structure from that of learning, NN-DFL uses the same inputs, the same differentiable episode simulation, loss, seeds and validation-based model selection, but none of the structure of UL_λ: the network outputs the delivery directly, $q_{it} = \bar{d}_i\,(1 + \delta_\theta(\phi_{it}))^+$ with $\bar{d}_i$ the store's mean daily demand, a store is visited whenever $q_{it} > 0$, and training starts from delivering one day of mean demand to every store every day. This is the type of neural ordering policy trained through simulated inventory dynamics by Alvo et al. [2]. Because it starts far from a good policy, it is given twice as many epochs as UL-DFL.

4.4 Integrated rolling-horizon policies

These policies solve the lookahead MILP of Section 3.2 every day with $H = 3$ ($H = 2$ when $m = 2$) and implement day 0. They differ only in the planning demand.

- **LA-Mean** uses the point forecasts $\hat{\mu}$: the classical predict-then-optimise policy.
- **LA-Quantile** uses $\hat{Q}_\alpha$ for every store and day. With α tuned on validation episodes, this is a parametric cost-function approximation in the sense of Powell [38]: a deterministic lookahead whose inputs are adjusted to perform well under uncertainty.
- **LA-Quantile2** uses $\hat{Q}_{\alpha_0}$ on the implemented day and $\hat{Q}_{\alpha_1}$ on all later lookahead days, with (α_0, α_1) tuned. It is an ablation of LA-DFL: it can represent the day-specific safety stock that LA-DFL learns (Section 6.2) but none of its context- or stock-dependent corrections.
- **LA-Robust** uses budgeted demand uncertainty [7]. Demand on lookahead day k ranges over $[\hat{\mu}_k, \hat{\mu}_k + \hat{\delta}_k]$ with $\hat{\delta}_k = \hat{Q}_{0.95} - \hat{\mu}_k$, and the cumulative scaled deviation of the first $k+1$ days is at most $\Gamma_k = \min\{\Gamma\sqrt{k+1}, k+1\}$. The planning path reproduces the worst-case cumulative demand of every prefix, which is the deterministic equivalent shown by Bertsimas and Thiele [7] for inventory systems and used in robust inventory routing [41]. Γ is tuned.
- **ScenPO-SS** is the Scenario Predict-then-Optimize method of Jia et al. [32]: ten demand scenarios are sampled from the multi-horizon quantile forecasts (each scenario draws a quantile band uniformly and then draws every store-day value uniformly inside that band), and the single-stage scenario model chooses one delivery and routing plan that minimises the average cost over the scenarios. Two adaptations are needed and disclosed: inventory follows the perishable age-structured dynamics with lost sales instead of the original non-perishable backlogging model, and the quantile model is LightGBM rather than an MQ-RNN, so that all policies use the same forecasts.
- **LA-DFL** (proposed) uses decision-focused planning targets (Section 4.5).
- **PI** solves the same lookahead with the realised demand of the next H days. It is a value-of-information reference, not a lower bound.

Table 2: Policies compared. All use the same observed stock and the same forecasts.

Policy	Family	Decision rule	Tuned on validation
BSP	Inventory rule, route second	Periodic base-stock on total stock	Cycle R, quantile α
BSP-EW	Inventory rule, route second	Base-stock plus estimated waste [9, 23]	R, α
UL_λ	Stochastic PIRP rule	Deliver-up-to-level [14]	λ, α, β
UL-DFL	Stochastic PIRP rule	Deliver-up-to-level with decision-focused order-up-to level (this paper)	Network weights, early stopping, seed
NN-DFL	Neural ordering policy (ablation)	Delivery output by a network trained like UL-DFL [2]	Network weights, early stopping, seed
LA-Mean	Rolling-horizon PIRP	Mean forecasts	–
LA-Quantile	Rolling-horizon PIRP (CFA)	Quantile forecasts	α
LA-Quantile2	Rolling-horizon PIRP (CFA)	Day-specific quantile forecasts	α_0, α_1
LA-Robust	Rolling-horizon PIRP	Budgeted uncertainty [7]	Γ
ScenPO-SS	Rolling-horizon stochastic PIRP	Scenario PTO [32]	– (10 scenarios)
LA-DFL	Rolling-horizon PIRP	Decision-focused targets (this paper)	Network weights, early stopping, seed
PI	Reference	Realised demand	–

4.5 Decision-focused planning targets (LA-DFL)

LA-DFL (proposed) keeps the deterministic lookahead, which is fast and easy to explain to planners, and learns how much each planning target should deviate from the forecast. For store i, planning day t and offset k the target is

$$\tilde{d}_{i,t+k}(\theta) = \Big(\hat{Q}_{\alpha^\star}(D_{i,t+k} \mid \xi) + \hat{\sigma}_{i,t+k}\, \delta_\theta(\phi_{itk})\Big)^+, \tag{10}$$

where $\alpha^\star$ is the quantile level selected for LA-Quantile, and δ_θ is a two-layer neural network whose output layer is initialised to zero, so that training starts exactly at the tuned LA-Quantile policy. The input ϕ_{itk} contains the forecast features, the scaled mean, scale and 5–95% range of the forecast, the offset k, and the store's current stock by remaining life. The correction can therefore depend on context (e.g. promotions, which raise both mean and variance) and on stock age, which neither the forecasts nor a single quantile level capture.

Learning signal. The visit pattern chosen by the MILP is piecewise constant in the targets, so its gradient carries no information. Given the pattern, however, deliveries are piecewise linear in the targets: each delivery covers the planned demand of its coverage window W_k, net of the usable stock projected by FIFO, capped by the remaining shelf room and scaled to the vehicle capacity. We implement this coverage map $q(\tilde{d}; V)$ as a differentiable layer and evaluate the realised H-day FIFO cost of the resulting deliveries under the observed demand with a differentiable simulator. Automatic differentiation then returns

$$\nabla_\theta\, C_{\text{FIFO}}\big(q(\tilde{d}_\theta; V), D\big) = \Big(\frac{\partial \tilde{d}}{\partial \theta}\Big)^\top \Big(\frac{\partial q}{\partial \tilde{d}}\Big)^\top \nabla_q C_{\text{FIFO}}, \tag{11}$$

whose last factor is the marginal value of a delivery bounded in Proposition 2, and whose stationary points are characterised in Proposition 3. In words: if, in hindsight, an extra unit in a

store's delivery would have prevented a lost sale, the targets of the days that delivery covered are pushed up; if the extra unit would have been wasted or carried, they are pushed down; and nothing changes when the store was not visited or capacity was binding.

Training. Training states are the decision epochs visited by the tuned LA-Quantile policy on training episodes. Each epoch re-solves the MILP at the current targets to refresh the visit patterns (a form of solution caching, 36), takes Adam steps on the average realised cost of minibatches, and evaluates the closed-loop cost of the current network on the validation episodes; the network with the lowest validation cost is kept, which may be the LA-Quantile starting point. Algorithm 1 summarises the procedure.

Algorithm 1 Training of decision-focused planning targets (LA-DFL)

Require: training states $\{(B^{(s)}, \xi^{(s)}, D^{(s)})\}$ from LA-Quantile runs; tuned level $\alpha^\star$; epochs E
initialise $\delta_\theta \equiv 0$; $\theta^\star \leftarrow \theta$; $J^\star \leftarrow$ validation cost of LA-Quantile
for $e = 1, \dots, E$ **do**
  for every state s: solve the lookahead MILP with targets $\tilde{d}_\theta^{(s)}$; store visit pattern $V^{(s)}$
  for each minibatch $\mathcal{B}$ **do**
    $L \leftarrow |\mathcal{B}|^{-1} \sum_{s \in \mathcal{B}} C_{\mathrm{FIFO}}\big(q(\tilde{d}_\theta^{(s)}; V^{(s)}), D^{(s)}\big)$ ▷ differentiable coverage layer + FIFO simulator
    $\theta \leftarrow \mathrm{Adam}(\theta, \nabla_\theta L)$
  end for
  $J \leftarrow$ closed-loop validation cost of LA-DFL with θ; **if** $J < J^\star$ **then** $\theta^\star \leftarrow \theta$, $J^\star \leftarrow J$
end for
return $\theta^\star$

The two learners differ in what the gradient sees. LA-DFL learns from the H-day cost of one plan with the visit pattern chosen by the MILP, so it can only correct the plan's targets; UL-DFL learns from the cost of whole episodes, so it can trade today's order against tomorrow's carried stock and waste, but it keeps the rule's simple visit trigger. LA-DFL needs an MILP solve per training state and epoch; UL-DFL needs none. Compared with the decision-focused learner of the original submission, three changes are essential. The learner no longer differentiates a heuristic surrogate of a MILP that was in practice never solved; the MILP is solved to a proven gap and only supplies visit patterns. The loss is the realised physical cost rather than regret against a heuristic full-information plan. And training and model selection are performed on the closed-loop system that is evaluated.

5 Experimental design

5.1 Network, products and costs

The base network consists of a depot and the first ten customers of Solomon's R101 instance [40], which are scattered around the depot at distances of 11 to 32; coordinates define Euclidean distances and time windows are ignored. Two further layouts use the first ten customers of C101, a tight cluster 15–21 distance units from the depot (*clustered*), and of RC101, a remote cluster of eight stores 33–40 units away plus two stores near the depot (*mixed*). Two vehicles serve the stores. Vehicle capacity is set through the fleet ratio $\rho = KQ/\bar{D}$, where $\bar{D}$ is mean total daily demand; the base case has $\rho = 1.25$ (117 units per vehicle), so that the fleet can carry a quarter more than average demand, as is common in grocery distribution where vehicles are sized for peak days. Shelf capacity is three days of mean store demand. The base product has a shelf life of $m = 3$ days. Costs are $h = 1$ per unit and night, $p_s = 5$ per expired unit and $p_u = 20$ per lost sale, with routing cost one per distance unit; per unit of demand, waste is five times and a lost sale twenty times as expensive as one night of holding.

5.2 Demand and information

Demand in the synthetic settings is generated by a contextual negative-binomial model (code in the replication package). Store base levels range from 10 to 22 units per day; demand varies with the day of the week (Saturday +30%), announced promotions (12% of store-days, discounts of 10–30%, multiplying demand by up to 1.8 and lowering the negative-binomial size parameter from 8 to 3.2, so that promotions raise both level and variability), temperature, holidays, and a latent store level following a log-AR(1) process with autocorrelation 0.92 that can only be inferred from recent sales. The coefficient of variation of daily store demand is about 0.4 on regular days. The temperature forecast error grows with the lead time. The model is designed to reproduce qualitative features of fresh retail demand (weekly pattern, promotion lifts, heteroscedasticity, persistence), not calibrated to a specific retailer; the real-data case below tests the policies on observed demand.

Each experiment uses 200 training, 50 validation and 100 test episodes, generated with disjoint random seeds. An episode consists of 35 days of history (to compute lagged features), 21 decision days, and four further days so that lookahead forecasts and the perfect-information reference are defined on the last decision days. Every policy starts each test episode with empty stores and is evaluated on the same 100 demand paths (common random numbers).

5.3 Real-data case

The real-data case replaces the synthetic demand by observed sales of FreshRetailNet-50K [45], a public dataset of 50,000 store–product series of a fresh-food retailer over 97 days (its 90-day training and seven-day evaluation parts combined), with hourly sales, hourly out-of-stock indicators, discounts, holidays and weather (CC BY 4.0 licence). An episode is one product sold at ten stores of the same city over the last 60 days of the data (35 history, 21 decision and four further days). We keep series without missing days and with fewer than 10% zero-sales days and form at most one ten-store group per city and product. Products, not groups, are assigned to the training, validation and test sets, so that no product appears in two of them: the 200 training, 50 validation and 100 test episodes come from 101, 28 and 56 products in 13 cities.

Three preparation steps are needed and disclosed. First, sales are censored on days with stock-outs, which concern about 40% of the store-days. Latent demand is estimated by dividing a day's sales by the share of the series' typical intraday sales profile that falls into in-stock hours, the profile being estimated on the series' fully in-stock days and the share floored at 0.25. Second, the products differ in volume by orders of magnitude, whereas the network, fleet and shelf capacities are those of the synthetic study; each store's latent demand is therefore rescaled to the store's base volume using the mean over the 35 history days only, so that no information from the decision days is used. Third, fleet and shelf capacities are dimensioned on the network's average volume, so groups whose sales level changes by more than $-20\%/+25\%$ between the history and decision windows (typically products entering or leaving their season) are excluded; in them the fleet, not the replenishment policy, would determine the outcome. This selection concerns only which data enter the experiment; forecasts and policies use past information only. In the resulting test episodes demand on the decision days is on average 6% above the history level, the coefficient of variation of daily store demand is 0.55, and discounts are offered on 44% of the store-days. Forecast features are those of the synthetic study, with the observed temperature used as its own forecast.

5.4 Configurations

Besides the base case we vary one factor at a time: network layout (clustered, mixed); shelf life $m \in \{2, 4\}$; lost-sales penalty $p_u \in \{10, 40\}$; fleet ratio $\rho \in \{1.10, 1.60\}$; routing cost $c^{\mathrm{r}} = 3$ per distance unit, which makes consolidation of deliveries more valuable; demand variability (negative-binomial size 3 instead of 8, i.e. coefficient of variation about 0.6 on regular days);

partial LIFO issuing, where 30% of customers take the freshest units; and the real-data case. In the LIFO configuration all policies are tuned and trained under FIFO and evaluated under mixed issuing, which tests robustness to a misspecified issuing assumption that practitioners often face.

5.5 Tuning, training and inference

All parameters are selected on validation episodes by closed-loop simulation: BSP and BSP-EW over $R \in \{1, 2\}$ and $\alpha \in \{0.6, 0.7, 0.8, 0.9, 0.95\}$; UL_λ over $\lambda \in \{1, 2, 3\}$, $\alpha \in \{0.7, 0.8, 0.9, 0.95\}$, $\beta \in \{0.5, 0.7, 0.9\}$; LA-Quantile over $\alpha \in \{0.5, \ldots, 0.9\}$; LA-Quantile2 over $\alpha_0 \in \{0.9, 0.95\}$ and $\alpha_1 \in \{0.6, 0.7, 0.8\}$; LA-Robust over $\Gamma \in \{0.25, 0.5, 1, 1.5\}$.

UL-DFL is trained on all 200 training episodes for 30 epochs with Adam (learning rate 10^{-3}, minibatches of ten episodes, gradient norm clipped at 5, L_2 penalty 10^{-4} on the output layer), from three random seeds in every configuration; NN-DFL is trained identically but for 60 epochs. LA-DFL is trained on the decision states visited by the tuned LA-Quantile policy in 100 training episodes (2,100 states), with eight epochs, visit patterns re-optimised every second epoch, Adam (learning rate $2 \cdot 10^{-3}$, minibatch 64), a two-layer network with 64 SiLU units, and three seeds in the base case and one elsewhere. For all learners the network and the seed with the lowest closed-loop validation cost are kept. NN-DFL was added after the primary hypotheses had been fixed; its comparisons are reported as secondary evidence. Test episodes are used once.

We prespecify two families of primary hypotheses in the base case. For UL-DFL they compare it with UL_λ, BSP-EW, ScenPO-SS, LA-DFL and LA-Quantile2; for LA-DFL with LA-Quantile, ScenPO-SS, LA-Robust, BSP-EW and UL_λ. The unit of analysis is the episode; for each comparison we report the mean paired cost difference, a 95% paired bootstrap interval (10,000 resamples), paired t and Wilcoxon signed-rank tests, and Holm-adjusted p-values within each family [26]. All other comparisons and configurations are reported with paired intervals as secondary evidence.

5.6 Optimisation settings

Lookahead MILPs are solved by HiGHS [28] through SciPy with a relative gap of 0.5% and time limits of 20 seconds (deterministic) and 60 seconds (scenario), with integer vehicle assignments on the implemented day and relaxed assignments on later lookahead days (Section 3.2). The LA-DFL oracle uses the same model with a 1% gap. If a time limit were reached the best incumbent would be used and its proven gap recorded; Section 6.8 reports solver statistics. All experiments run on a 16-core laptop computer (Intel Core Ultra 9 285H) with 13 to 15 parallel worker processes; the code, seeds and configuration files are in the replication package.

6 Results

6.1 Base case

Table 3 reports test-set averages over the 100 episodes and Figure 1 decomposes the costs. Tuning selected order-up-to levels at the 0.95 quantile with daily delivery for BSP and BSP-EW, UL_λ with $\lambda = 1$, $\alpha = 0.95$, $\beta = 0.9$, quantile level $\alpha^\star = 0.9$ for LA-Quantile, levels $(0.9, 0.8)$ for LA-Quantile2 and budget $\Gamma = 0.5$ for LA-Robust.

Table 3: Base case: mean cost per 21-day test episode and its components, fill rate, waste rate (expired units per delivered unit) and store visits per day (100 episodes).

Policy	Total cost (SD)	Routing	Holding	Waste	Lost sales	Fill rate (%)	Waste rate (%)	Visits/day
BSP	10,851 (2,050)	4,391	2,934	471	3,055	96.2	2.48	9.5
BSP-EW	10,885 (2,018)	4,409	2,980	474	3,022	96.3	2.49	9.6
UL_λ	10,799 (2,099)	4,322	2,939	481	3,057	96.2	2.53	9.2
UL-DFL	10,708 (2,038)	4,337	2,953	379	3,038	96.2	1.98	9.4
NN-DFL	10,871 (2,065)	4,387	2,976	361	3,148	96.1	1.89	9.6
LA-Mean	19,371 (2,715)	3,961	822	27	14,561	81.2	0.17	9.7
LA-Quantile	12,122 (2,211)	3,516	4,026	1,161	3,418	95.7	5.66	7.8
LA-Quantile2	11,433 (2,234)	3,741	3,072	581	4,039	94.9	2.97	8.6
LA-Robust	11,948 (2,198)	3,835	2,305	303	5,504	93.0	1.62	9.0
ScenPO-SS	11,702 (1,750)	3,425	3,762	1,001	3,515	95.6	5.03	7.7
LA-DFL	11,548 (2,286)	3,748	3,244	607	3,948	95.1	3.05	8.7
PI	5,758 (2,227)	4,126	132	0	1,500	98.3	0.00	9.8

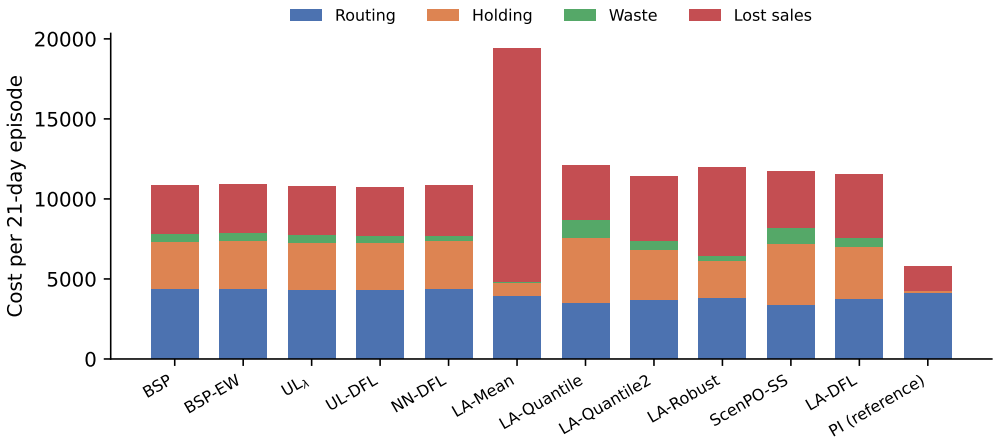


Figure 1: Cost composition of each policy in the base case.

Four observations stand out. First, uncertainty is expensive: with perfect demand information the same lookahead costs 5,758, about half of the best implementable policy, almost entirely because lost sales and holding vanish. Second, point forecasts are a poor input for perishable replenishment. LA-Mean plans to sell exactly the expected demand and loses 14,561 per episode in lost sales (fill rate 81.2%), as Proposition 2 suggests when p_u is four times p_s. Third, among the policies without learning, the three stock-age-based inventory rules are the cheapest: UL_λ, BSP and BSP-EW cost between 10,799 and 10,885 with a fill rate of 96.2% and a waste rate of about 2.5%. With daily review, little of the current stock is at risk of expiring before it is sold, so the waste correction of BSP-EW rarely changes an order.

Fourth, integration with routing does reduce travel but loses more on the inventory side. The uncertainty-aware lookahead policies visit 7.7–9.0 stores per day instead of 9.2–9.6 and save about 500–1,000 in routing, but LA-Quantile and ScenPO-SS hold 26–37% more stock than the rules and waste about twice as much (waste rates 5.03% and 5.66%), while LA-Robust avoids the waste at the price of a fill rate of 93.0%. A deterministic plan that covers two or three days with quantile targets stacks a safety buffer on every covered day, even though the next day's re-planning will correct any shortfall. LA-Quantile2, which keeps the 0.9 quantile only for the implemented day and plans later days at the 0.8 quantile, halves the waste rate of LA-Quantile (2.97% vs. 5.66%) and costs 5.7% less; the scenario model reduces lost sales but, because its single plan must work for all scenarios, it also covers several days generously.

Table 4: Paired episode-level comparisons of UL-DFL with every other policy (base case, 100 episodes). Negative differences favour UL-DFL. Holm adjustment over the five prespecified primary comparisons of UL-DFL.

Comparison	Mean diff.	95% bootstrap CI	Rel. (%)	Wins (%)	p (t)	p (Wilcoxon)	Holm p
UL-DFL − BSP	-142.9	[-188.7, -96.2]	-1.3	69	<0.001	<0.001	–
UL-DFL − BSP-EW	-177.2	[-224.3, -129.6]	-1.7	77	<0.001	<0.001	<0.001
UL-DFL − UL$_\lambda$	-90.7	[-134.1, -46.5]	-0.8	68	<0.001	<0.001	<0.001
UL-DFL − NN-DFL	-162.7	[-212.0, -112.1]	-1.5	79	<0.001	<0.001	–
UL-DFL − LA-Mean	-8,662.6	[-8,980.4, -8,348.3]	-44.7	100	<0.001	<0.001	–
UL-DFL − LA-Quantile	-1,413.4	[-1,576.2, -1,252.6]	-11.5	96	<0.001	<0.001	–
UL-DFL − LA-Quantile2	-724.7	[-854.1, -591.8]	-6.1	88	<0.001	<0.001	<0.001
UL-DFL − LA-Robust	-1,239.7	[-1,361.4, -1,117.1]	-10.3	96	<0.001	<0.001	–
UL-DFL − ScenPO-SS	-994.2	[-1,177.5, -795.7]	-8.7	87	<0.001	<0.001	<0.001
UL-DFL − LA-DFL	-839.7	[-972.4, -705.7]	-7.0	94	<0.001	<0.001	<0.001
UL-DFL − PI	4,950.2	[4,786.8, 5,109.4]	95.0	0	<0.001	<0.001	–

UL-DFL is the cheapest implementable policy (Table 4). It costs 0.8% less than the tuned UL$_\lambda$ from which it starts (95% CI [-134, -47], lower in 68% of the episodes), 1.7% less than BSP-EW, 8.7% less than ScenPO-SS, 6.1% less than LA-Quantile2 and 7.0% less than LA-DFL; all five primary comparisons are significant after Holm adjustment ($p < 0.001$). The gain over UL$_\lambda$ is a waste reduction: at the same fill rate (96.2%) and almost the same lost-sales cost, UL-DFL lowers the waste rate from 2.53% to 1.98%, i.e. it throws away a fifth fewer units, while holding and routing costs rise by at most 0.5%.

Table 5: Paired episode-level comparisons of LA-DFL with every other policy (base case, 100 episodes). Negative differences favour LA-DFL. Holm adjustment over the five prespecified primary comparisons of LA-DFL.

Comparison	Mean diff.	95% bootstrap CI	Rel. (%)	Wins (%)	p (t)	p (Wilcoxon)	Holm p
LA-DFL − BSP	696.8	[551.8, 844.2]	6.4	18	<0.001	<0.001	–
LA-DFL − BSP-EW	662.5	[517.0, 810.7]	6.0	21	<0.001	<0.001	<0.001
LA-DFL − UL$_\lambda$	749.0	[606.5, 891.8]	7.0	14	<0.001	<0.001	<0.001
LA-DFL − UL-DFL	839.7	[705.7, 972.4]	7.8	6	<0.001	<0.001	–
LA-DFL − NN-DFL	677.0	[556.9, 797.0]	6.2	13	<0.001	<0.001	–
LA-DFL − LA-Mean	-7,822.9	[-8,167.1, -7,477.1]	-40.4	100	<0.001	<0.001	–
LA-DFL − LA-Quantile	-573.7	[-683.7, -465.2]	-4.9	84	<0.001	<0.001	<0.001
LA-DFL − LA-Quantile2	115.0	[41.2, 190.6]	1.0	41	0.004	0.015	–
LA-DFL − LA-Robust	-400.0	[-542.4, -255.7]	-3.4	70	<0.001	<0.001	<0.001
LA-DFL − ScenPO-SS	-154.5	[-365.6, 67.9]	-1.6	59	0.167	0.124	0.167
LA-DFL − PI	5,789.9	[5,575.4, 6,009.5]	110.2	0	<0.001	<0.001	–

LA-DFL improves the deterministic lookahead from which it starts (Table 5). It reduces cost by 4.9% relative to LA-Quantile (95% CI [-684, -465], lower in 84% of the episodes) and by 3.4% relative to LA-Robust; it is 1.6% cheaper than ScenPO-SS, which is not significant (Holm-adjusted $p = 0.167$). It does so by halving the waste rate of LA-Quantile (3.05% vs. 5.66%) while accepting somewhat more lost sales. The simple LA-Quantile2, however, is 1.0% cheaper still (CI [41, 191]): the gain of LA-DFL over LA-Quantile is captured by a day-specific quantile level, which LA-DFL discovered (Section 6.2) but did not improve on. LA-DFL also remains 6.0% more expensive than BSP-EW and 7.0% more expensive than UL$_\lambda$, both significant.

6.2 What the decision-focused corrections learn

Table 6 summarises the learned corrections of UL-DFL on the delivery decisions it takes in the 100 base-case test episodes (the rule triggers a delivery on 94% of the store-days; on the others the correction has no effect). The average correction is nearly zero: the tuned 0.95 level is right

on average. What the network learns is *when* to deviate from it, along two dimensions that a one-day order-up-to rule cannot see. On promotion days it lowers the level to an implied service level of about 0.89, i.e. by about a third of a day of mean demand. Promotions raise the forecast spread, so the 0.95 quantile lies far above the expected demand, and whatever is left when the promotion ends meets regular demand and ages. Over the week it raises the level from Wednesday to Friday (implied levels 0.96–0.97) and lowers it from Saturday to Monday (0.92–0.93): a unit left over before the weekend is sold in the Saturday peak, whereas a unit left over after it meets lower demand for the rest of its shelf life. The age of the stock on hand matters little, because the rule's order quantity already nets out the stock projected to expire. Both corrections reduce the stock that is still on the shelf when demand falls, which is where the waste of the rule arises.

Table 6: Learned corrections of UL-DFL on the delivery decisions of 100 test episodes (base case): mean correction δ_θ of the order-up-to level in units of the forecast scale, implied service level $\Phi(z_{0.95} + \delta_\theta)$, and change of the order-up-to level in days of mean store demand. Stock is measured in days of mean store demand.

Delivery decisions	n	Correction δ_θ (SD)	Implied service level	Change of S (days)
all delivery decisions	19,650	+0.00 (0.28)	0.944	-0.02
no promotion	17,080	+0.07 (0.23)	0.952	+0.03
promotion	2,570	-0.40 (0.22)	0.887	-0.31
Mon	2,810	-0.14 (0.20)	0.931	-0.07
Tue	2,758	-0.01 (0.22)	0.945	-0.02
Wed	2,835	+0.18 (0.21)	0.962	+0.06
Thu	2,828	+0.28 (0.23)	0.969	+0.11
Fri	2,856	+0.12 (0.22)	0.957	+0.04
Sat	2,922	-0.24 (0.18)	0.917	-0.14
Sun	2,641	-0.17 (0.20)	0.925	-0.09
stock <0.25 d	4,376	+0.02 (0.27)	0.946	-0.01
stock 0.25-0.5 d	2,599	+0.01 (0.27)	0.945	-0.01
stock >0.5 d	12,675	-0.00 (0.28)	0.943	-0.02
no expiring units	13,355	+0.02 (0.28)	0.945	-0.01
units expiring today	6,295	-0.03 (0.28)	0.940	-0.03

Table 7 reports the corrections of LA-DFL on the decision states of 20 test episodes. Here the dominant pattern is temporal within the plan. For the implemented day the network keeps the target at about the 0.91 quantile, essentially the tuned LA-Quantile level, but it lowers the targets of the second and third lookahead days to about the 0.80 and 0.77 quantiles. In inventory terms, the network has learned that only today's shortfall is irreversible: tomorrow's planned demand will be re-planned with tomorrow's stock and forecast, so a full safety buffer on that day mostly turns into carried stock and, with a three-day shelf life, into waste. This is precisely the recourse that a deterministic lookahead ignores and that Proposition 3 lets the gradient account for. Context-specific corrections are small: targets are lowered somewhat more on promotion days and slightly less when the store holds more or older stock. The day-specific levels learned by LA-DFL are those that the tuning of LA-Quantile2 selects, $(0.9, 0.8)$, which explains why the two policies perform alike.

Table 7: Learned corrections of LA-DFL on the decision states of 20 test episodes (base case): mean correction δ_θ in forecast-scale units, implied quantile level of the planning target, and target relative to the mean forecast. k is the lookahead offset; stock is measured in days of mean store demand.

Targets	n	Correction δ_θ (SD)	Implied quantile level	Target / mean forecast
all	12,600	-0.37 (0.38)	0.826	1.46
k=0	4,200	+0.12 (0.15)	0.910	1.67
k=1	4,200	-0.53 (0.15)	0.804	1.38
k=2	4,200	-0.69 (0.17)	0.765	1.31
no promotion	11,078	-0.36 (0.39)	0.828	1.45
promotion	1,522	-0.47 (0.32)	0.818	1.53
stock <0.25 d	2,694	-0.42 (0.39)	0.816	1.43
stock 0.25-0.75 d	4,059	-0.40 (0.38)	0.821	1.45
stock >0.75 d	5,847	-0.33 (0.38)	0.835	1.48
no expiring units	8,088	-0.39 (0.38)	0.822	1.45
units expiring today	4,512	-0.33 (0.38)	0.834	1.48

For both learners the closed-loop validation cost improved during the first three to five epochs and deteriorated afterwards (Figure 2), which is why model selection on closed-loop validation cost is part of the method. The result does not depend on a lucky seed (Table 8): the three UL-DFL networks are 0.8–0.9% cheaper than UL_λ on the base-case test set, and the three LA-DFL networks 3.5–4.7% cheaper than LA-Quantile.

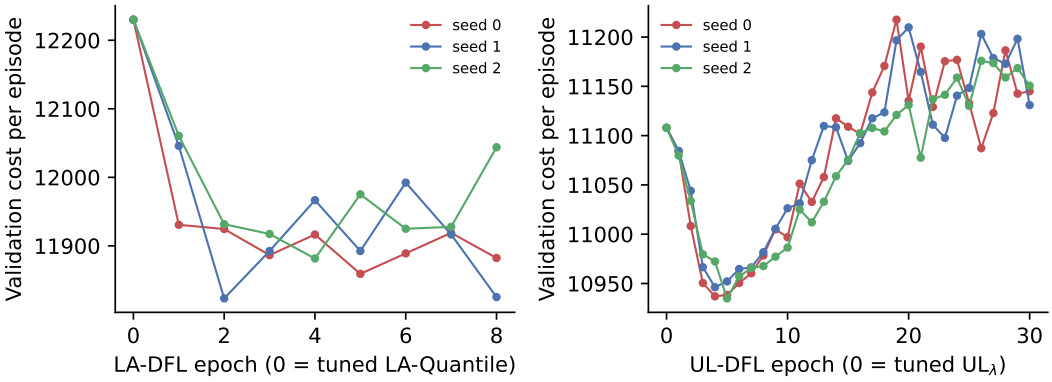


Figure 2: Closed-loop validation cost by training epoch and seed (base case). Left: LA-DFL, epoch 0 is the tuned LA-Quantile policy. Right: UL-DFL, epoch 0 is the tuned UL_λ rule.

Table 8: Test-cost difference (%) of each trained network relative to the policy it starts from, by seed; "selected" is the network chosen on validation cost.

Setting	Comparison	Selected (%)	Seed 0	Seed 1	Seed 2
Base	UL-DFL vs UL_λ	-0.84	-0.89	-0.94	-0.84
Base	NN-DFL vs UL_λ	+0.67	+0.64	+0.55	+0.67
Base	LA-DFL vs LA-Quantile	-4.73	-3.59	-4.73	-3.48
Clustered	UL-DFL vs UL_λ	-1.29	-1.29	-1.30	-1.26
Clustered	NN-DFL vs UL_λ	-0.14	-0.08	-0.14	+0.06
Mixed	UL-DFL vs UL_λ	-1.07	-1.07	-1.07	-1.00
Mixed	NN-DFL vs UL_λ	+0.22	+0.17	-0.04	+0.22
$m = 2$	UL-DFL vs UL_λ	-0.89	-0.82	-0.80	-0.89
$m = 2$	NN-DFL vs UL_λ	-0.92	-1.01	-0.95	-0.92
$m = 4$	UL-DFL vs UL_λ	-0.14	-0.17	-0.13	-0.14
$m = 4$	NN-DFL vs UL_λ	+1.89	+1.79	+1.87	+1.79
$p_u = 10$	UL-DFL vs UL_λ	-1.00	-0.95	-1.00	-0.92
$p_u = 10$	NN-DFL vs UL_λ	+1.07	+1.19	+1.17	+1.07
$p_u = 40$	UL-DFL vs UL_λ	-0.92	-1.27	-0.92	-1.33
$p_u = 40$	NN-DFL vs UL_λ	+0.17	+0.17	+0.18	+0.64
$\rho = 1.10$	UL-DFL vs UL_λ	-0.74	-0.74	-0.81	-0.81
$\rho = 1.10$	NN-DFL vs UL_λ	+0.22	-0.07	+0.22	+0.32
$\rho = 1.60$	UL-DFL vs UL_λ	-0.45	-0.45	-0.40	-0.47
$\rho = 1.60$	NN-DFL vs UL_λ	+0.95	+1.09	+1.17	+0.95
High var.	UL-DFL vs UL_λ	-0.27	-0.20	-0.18	-0.27
High var.	NN-DFL vs UL_λ	+0.60	+0.46	+0.65	+0.60
$c^r = 3$	UL-DFL vs UL_λ	-0.83	-0.83	-0.45	-0.66
$c^r = 3$	NN-DFL vs UL_λ	+4.18	+4.28	+4.24	+4.18
30% LIFO	UL-DFL vs UL_λ	-0.75	-0.80	-0.78	-0.75
30% LIFO	NN-DFL vs UL_λ	+1.01	+0.90	+1.24	+1.01
30% LIFO	LA-DFL vs LA-Quantile	-4.77	-4.35	-4.77	-3.32
Real data	UL-DFL vs UL_λ	-0.58	-0.54	-0.83	-0.58
Real data	NN-DFL vs UL_λ	+0.51	+0.78	+0.51	+0.78

6.3 Can re-tuning the rule reproduce UL-DFL?

A natural objection is that UL-DFL merely finds better values of the rule's parameters than the tuning grid of Section 5. Figure 3 places UL-DFL in the waste–service plane together with 124 parameterisations of the rules, all evaluated on the base-case test episodes: UL_λ with $\lambda = 1$, ten service levels α between 0.85 and 0.99 and eight trigger levels β between 0.5 and 0.99, UL_λ with $\lambda \in \{2, 3\}$ on the original grid, and daily BSP and BSP-EW at the same ten service levels. No parameterisation serves at least UL-DFL's fill rate (96.2%) with no more waste: at that fill rate the least waste any rule achieves is 2.48% of the delivered units, against 1.98% for UL-DFL. The learned corrections move the policy beyond the waste–service frontier of the rules, which no choice of a single service level and trigger can do.

In total cost the comparison is closer. The cheapest of the 124 rules in hindsight—selected on the test episodes themselves, an advantage UL-DFL does not have—is UL_λ with $\alpha = 0.96$ and $\beta = 0.8$. It costs 10,715 per episode, the same as UL-DFL (difference -0.05%, 95% CI $[-59, 46]$), but it gets there differently: its lower trigger saves routing (4,168 vs. 4,337), and it wastes 2.80% of the delivered units, 41% more than UL-DFL. Part of the 0.8% cost advantage of UL-DFL over the tuned rule therefore reflects the resolution of the tuning grid; the waste reduction at a given service level does not. The real-data case gives the same picture: no rule parameterisation reaches UL-DFL's fill rate (93.8%) with its waste rate (2.96%; the least any rule achieves at that fill rate is 3.15%), and the best rule in hindsight ($\alpha = 0.96$, $\beta = 0.85$) matches its cost (+0.02% for UL-DFL, 95% CI $[-50, 78]$) while wasting 23% more.

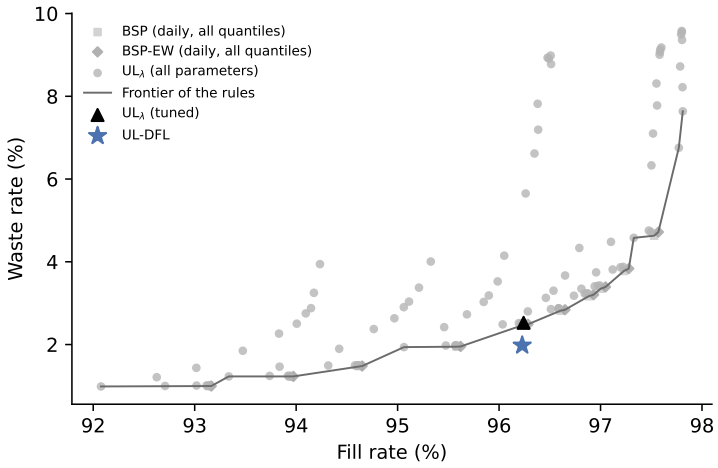


Figure 3: Waste rate against fill rate of 124 parameterisations of the rules and of UL-DFL (base case, 100 test episodes). The line connects the rule parameterisations that are not dominated in both measures.

6.4 Does the learner need the rule?

The converse question is whether the rule contributes anything once a network is trained on realised cost. NN-DFL answers it: it has the same inputs, differentiable simulation, loss, seeds and model selection as UL-DFL, but outputs the delivery directly, without the visit trigger and order-up-to structure of UL$_\lambda$. In the base case it costs 10,871 per episode, 1.5% more than UL-DFL (95% CI of UL-DFL minus NN-DFL [-212, -112]) and more than the tuned UL$_\lambda$ it would replace. It does learn to waste little (waste rate 1.89%), but it loses more sales and, lacking a trigger, visits slightly more stores (9.6 per day against 9.4). Across the thirteen configurations UL-DFL is significantly cheaper than NN-DFL in twelve, by 0.8–4.9% (Table 10). The gap is largest with expensive routing, where the trigger of UL-DFL cuts store visits to 7.1 per day while NN-DFL keeps visiting 9.6, and the two tie with a two-day shelf life, where frequent small deliveries suit the short shelf life. NN-DFL is more expensive than UL$_\lambda$ itself in eleven of the thirteen configurations (Table 8). These comparisons are secondary, because the ablation was added after the primary hypotheses were fixed, but they are consistent: with the same network and learning signal, the gain comes from placing the learner on the rule.

6.5 Sensitivity analysis

Table 9 reports test costs across the configurations, and Tables 10 and 11 the paired differences of the two learned policies. Five patterns emerge across the thirteen configurations. (i) UL-DFL is cheaper than UL$_\lambda$ in every configuration and significantly so in eleven, by 0.5–1.2%; all three seeds improve on UL$_\lambda$ in every configuration (Table 8). It is the cheapest implementable policy in eleven configurations, ties with NN-DFL for a two-day shelf life (difference 0.0%, 95% CI $[-45, 55]$), and is 2.9–10.3% below the best integrated policy in all configurations except the one with expensive routing. Its fill rate stays within 0.7 points of that of UL$_\lambda$, and in twelve configurations it wastes 6–38% fewer units; only with a two-day shelf life does it instead raise the fill rate (from 94.1% to 94.8%) at somewhat more waste. The two configurations without a significant gain are instructive. With a four-day shelf life UL$_\lambda$ wastes only 0.68% of the delivered units, so the corrections have little to remove: UL-DFL cuts waste by 38% but gives up 0.4 points of fill rate, and the net saving is 0.1% (95% CI [-53, 24]). With high demand variability the waste saving (16%) is largely offset by additional lost sales, a net 0.4% (CI [-103, 24]). The value of learning the order-up-to level is therefore largest when perishability makes the rule's single

service level costly but demand is predictable enough from context to act on it. (ii) Without learning, the stock-age-based rules remain cheaper than every integrated policy except with expensive routing: the best rule is 2.0–9.3% below the best integrated policy. The advantage is largest in the clustered layout, where all stores are close to each other, so that skipping a store saves little travel, and smallest with a low lost-sales penalty ($p_u = 10$), where the high service level of the rules is worth less, and with a loose or tight fleet (2.0–2.9%). Partial LIFO issuing raises the waste rate of every policy, by 40–90%, but does not change which policies are best. (iii) With three times higher routing cost, integration pays: ScenPO-SS is the cheapest policy, 2.7% below UL-DFL and 3.5% below UL_λ, and daily base-stock (BSP-EW) becomes more expensive than every uncertainty-aware lookahead. UL-DFL is still the cheapest rule, because the trigger of UL_λ already skips stores whose stock suffices. (iv) Among the integrated policies, LA-Quantile2 is best in eight configurations and ScenPO-SS in the other five: when lost sales are expensive ($p_u = 40$), the shelf life is two days, the fleet is tight, routing is expensive, and on the real data. LA-Mean is the most expensive policy in all thirteen. (v) LA-DFL is significantly cheaper than LA-Quantile in seven configurations (by 1.7–5.0%), not significantly different in five and more expensive with a two-day shelf life, and it is more expensive than LA-Quantile2 in every configuration (by 0.3–5.7%, significant in nine): on top of the lookahead, the value of decision-focused learning lies in revealing that the safety stock should depend on the planning day, which a two-parameter policy then implements.

Table 9: Mean test cost per episode by configuration (one factor changed at a time; bold: lowest implementable policy), and fill rates of selected policies.

Policy	Base	Clustered	Mixed	$m = 2$	$m = 4$	$p_u = 10$	$p_u = 40$	$\rho = 1.10$	$\rho = 1.60$	High var.	$c^r = 3$	30% LIFO	Real data
BSP	10,851	8,266	10,206	12,607	10,490	9,056	13,480	12,221	9,868	15,011	19,633	11,208	13,068
BSP-EW	10,885	8,289	10,235	12,633	10,490	9,082	13,496	12,227	9,922	15,089	19,704	11,256	13,090
UL_λ	10,799	8,274	10,199	12,599	10,339	8,956	13,374	12,152	9,798	14,885	18,856	11,141	13,027
UL-DFL	**10,708**	**8,168**	**10,090**	12,487	**10,324**	**8,867**	**13,251**	**12,063**	**9,755**	**14,845**	18,700	**11,057**	**12,951**
NN-DFL	10,871	8,263	10,221	**12,483**	10,534	9,052	13,396	12,179	9,892	14,974	19,645	11,253	13,093
LA-Mean	19,371	17,051	18,362	19,769	19,350	12,074	33,960	19,362	19,407	25,609	24,289	19,400	21,030
LA-Quantile	12,122	9,735	11,520	13,954	11,289	9,314	15,490	13,327	10,405	16,398	19,157	12,574	14,478
LA-Quantile2	11,433	9,110	10,830	13,553	10,977	9,136	15,436	13,047	10,071	15,631	18,630	11,787	14,185
LA-Robust	11,948	9,545	11,209	13,622	11,221	9,173	15,295	13,291	10,184	15,851	18,983	12,110	14,560
ScenPO-SS	11,702	9,544	11,315	13,179	11,051	9,393	14,501	12,512	10,997	16,218	**18,197**	12,162	13,776
LA-DFL	11,548	9,294	11,089	14,295	11,015	9,258	15,498	13,260	10,146	16,101	19,058	11,974	14,333
PI (reference)	5,758	3,352	5,180	6,121	5,752	5,005	7,260	7,678	4,728	7,097	13,589	5,758	7,329
Fill rate (%), UL-DFL	96.2	96.2	96.2	94.8	96.6	94.6	97.1	94.8	97.1	93.1	95.9	96.2	93.8
Fill rate (%), LA-DFL	95.1	95.2	95.1	93.1	95.4	93.4	95.8	92.8	96.3	91.9	94.9	94.9	92.1
Fill rate (%), ScenPO-SS	95.6	96.1	95.7	95.0	95.7	92.0	97.1	93.8	96.0	92.0	94.2	95.5	93.1
Fill rate (%), LA-Quantile	95.7	96.2	96.1	91.2	95.9	92.7	95.8	91.6	96.3	89.1	93.0	95.6	92.8
Fill rate (%), BSP-EW	96.3	96.3	96.3	94.5	96.3	94.7	97.0	94.5	97.4	93.8	96.3	96.2	93.8

Table 10: Paired cost difference of UL-DFL relative to each comparator, in percent of the comparator's cost, by configuration. Bold: 95% paired bootstrap interval excludes zero.

UL-DFL	Base	Clustered	Mixed	$m = 2$	$m = 4$	$p_u = 10$	$p_u = 40$	$\rho = 1.10$	$\rho = 1.60$	High var.	$c^r = 3$	30% LIFO	Real data
vs UL_λ	**-0.8**	**-1.2**	**-1.0**	**-0.8**	-0.1	**-1.0**	**-1.1**	**-0.9**	**-0.5**	-0.4	**-0.8**	**-0.7**	**-0.5**
vs BSP-EW	**-1.7**	**-1.5**	**-1.5**	**-1.2**	**-1.5**	**-2.4**	**-1.8**	**-1.4**	**-1.7**	**-1.8**	**-5.2**	**-1.8**	**-1.1**
vs ScenPO-SS	**-8.7**	**-14.9**	**-11.0**	**-5.6**	**-6.7**	**-5.6**	**-9.2**	**-4.1**	**-11.1**	**-8.7**	**+2.7**	**-9.2**	**-6.4**
vs LA-DFL	**-7.0**	**-11.4**	**-8.5**	**-11.7**	**-6.0**	**-4.1**	**-14.2**	**-8.9**	**-3.7**	**-7.8**	**-1.9**	**-7.2**	**-10.1**
vs LA-Quantile2	**-6.1**	**-9.9**	**-6.1**	**-7.0**	**-5.8**	**-2.9**	**-13.9**	**-7.3**	**-3.0**	**-5.0**	+0.4	**-5.8**	**-8.9**
vs NN-DFL	**-1.5**	**-1.1**	**-1.3**	+0.0	**-2.0**	**-2.1**	**-1.0**	**-1.0**	**-1.4**	**-0.8**	**-4.9**	**-1.7**	**-1.4**

Table 11: Paired cost difference of LA-DFL relative to each comparator, in percent of the comparator's cost, by configuration. Bold: 95% paired bootstrap interval excludes zero.

LA-DFL	Base	Clustered	Mixed	$m = 2$	$m = 4$	$p_u = 10$	$p_u = 40$	$\rho = 1.10$	$\rho = 1.60$	High var.	$c^r = 3$	30% LIFO	Real data
vs LA-Quantile	**-4.9**	**-5.0**	**-3.9**	**+2.1**	**-2.5**	-0.7	+0.0	-0.8	**-2.4**	**-1.7**	-0.4	**-4.9**	-1.4
vs LA-Quantile2	**+1.0**	**+1.9**	**+2.8**	**+5.7**	+0.3	**+1.4**	+0.7	**+1.8**	+0.8	**+3.3**	**+2.4**	**+1.5**	+1.5
vs ScenPO-SS	-1.6	-3.4	**-2.4**	**+7.7**	-0.5	**-1.4**	**+6.4**	**+5.6**	**-7.6**	-0.8	**+4.8**	-1.9	**+4.5**
vs LA-Robust	**-3.4**	**-2.9**	-1.1	**+4.8**	**-1.8**	**+0.9**	+1.9	-0.2	-0.3	**+1.8**	+0.5	-1.2	-0.6
vs BSP-EW	**+6.0**	**+11.8**	**+8.1**	**+12.7**	**+5.1**	**+1.9**	**+15.2**	**+8.6**	**+2.2**	**+6.8**	**-3.3**	**+6.2**	**+10.5**
vs UL_λ	**+7.0**	**+12.1**	**+8.6**	**+13.1**	**+6.6**	**+3.5**	**+16.0**	**+9.2**	**+3.5**	**+8.4**	**+1.3**	**+7.4**	**+11.2**

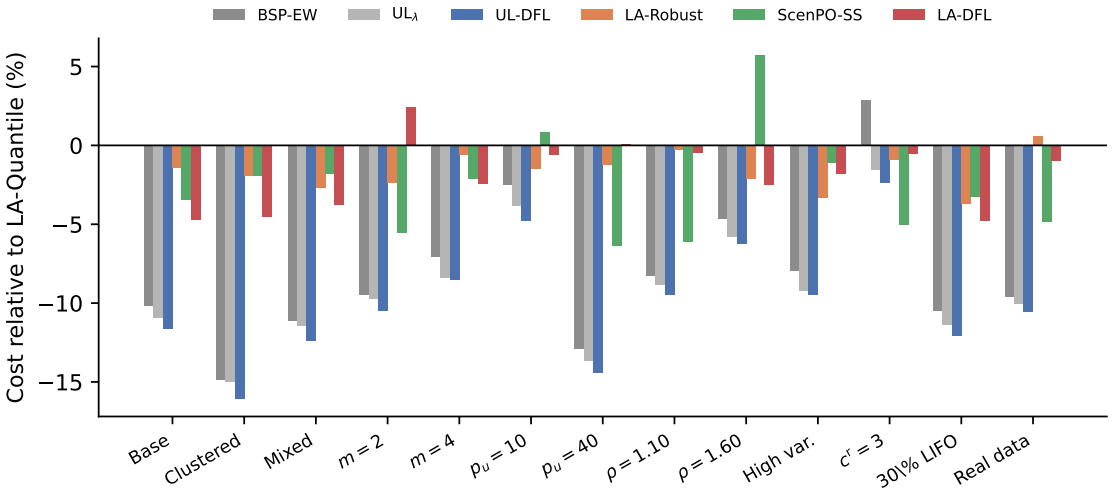


Figure 4: Cost of selected policies relative to LA-Quantile, by configuration (LA-Mean is omitted for readability).

6.6 Real-data case

On observed demand (column "Real data" of Tables 9 and 10) the ranking of the policy families is the same as in the synthetic base case. UL-DFL is the cheapest policy (12,951 per episode), 0.5% below UL$_\lambda$ (95% CI [-140, -13], Holm-adjusted $p = 0.022$), 1.1% below BSP-EW and 6.4% below ScenPO-SS, the best integrated policy. Without learning, the rules (13,027–13,090) are again cheaper than every integrated policy, and LA-Mean is again the most expensive. Service is lower than in the synthetic study for all policies (fill rate 93.6% for UL$_\lambda$), because observed demand is more volatile and its promotion and weekend peaks occasionally exceed the daily fleet capacity.

The saving of UL-DFL has a different composition on real data. It reduces lost sales by 2.3% and waste by 4.6% while holding 3.1% more stock, and its fill rate rises from 93.6% to 93.8%. The learned corrections explain why. Demand in these data peaks at the weekend (Saturday and Sunday are 16% and 29% above the average day of the history window, Wednesday to Friday 8–10% below it), and the network raises the order-up-to level on Thursday and Friday (implied service levels 0.96 and 0.99, Friday by 0.3 days of mean demand) and lowers it on Saturday to Monday (0.92, 0.85 and 0.92). Units delivered on Friday are carried into the weekend peak, which a one-day level misses, and units left over on Sunday meet the weaker start of the week. This is the same rule as in the synthetic data—build stock before the weekly peak, cut it on the days after—but here the peak is larger and falls on different days, so the correction mostly prevents weekend stock-outs instead of removing waste. Discounts, which are offered on 44% of the store-days in these data and are mostly small, have no systematic effect on the correction. LA-DFL is 1.4% cheaper than LA-Quantile, which is not significant (Holm-adjusted $p = 0.151$), and LA-Quantile2 is again cheaper than LA-DFL, though not significantly.

6.7 Store-level service

Average fill rates hide how service is distributed over the network (Figure 5). In all three layouts the inventory rules, with or without learning, serve every store at least 95.4%. The deterministic lookahead policies do not: in the base layout their worst-served store, one of the more distant stores, reaches only 89.9–90.6% (LA-Quantile, LA-Quantile2, LA-Robust, LA-DFL), and in the clustered and mixed layouts 89.6–94.5%. Consolidating deliveries saves the most travel at stores that are expensive to reach, and the plan then accepts shortages there; in the base layout the

store fill rate of the lookahead policies falls with distance from the depot (correlation −0.24 to −0.26 for LA-Quantile, LA-Quantile2 and LA-DFL), whereas that of the rules rises with it (+0.81). In the clustered and mixed layouts, where most stores are about equally far from the depot, the link with distance is weaker but the gap in the worst-served store remains. ScenPO-SS spreads service more evenly (worst store at least 95.0% in all three layouts). This effect is relevant for retailers with service-level agreements per store: an integrated plan may concentrate stock-outs at individual stores even when its average fill rate is high.

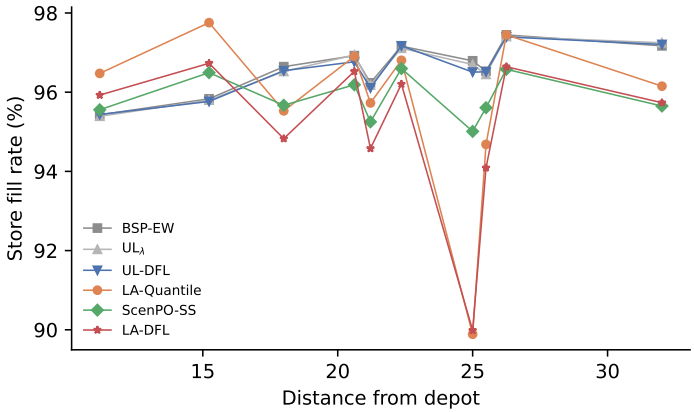


Figure 5: Store fill rate against distance from the depot (base case, mean over 100 test episodes).

6.8 Computation

All 14,700 lookahead decisions of the base-case test were solved to a proven gap of at most 0.50%, with 0 time-limit stops and 0 fallbacks; the same holds in every configuration reported. With 15 decisions solved in parallel, a deterministic lookahead decision takes about 2.40 seconds and a ScenPO-SS decision 5.15 seconds; the rules, with or without learned corrections, need milliseconds. Training one UL-DFL network (30 epochs over 200 episodes, including closed-loop validation after every epoch) took about two minutes; training one LA-DFL network (2,100 states, eight epochs including MILP re-solves and closed-loop validation) about 45 minutes. The complete base case, including tuning of all policies and three seeds of each learner, took 4.2 hours, and each other configuration 1.0–2.6 hours with 13–15 parallel workers (training three NN-DFL networks adds about ten minutes).

The lookahead policies are deterministic: solving the same decision twice returns the same plan. Their closed-loop cost is nevertheless sensitive to negligible changes in the planning input, because several plans lie within the 0.5% optimality gap and the solver returns the first one it proves; a different plan on one day changes the stock and therefore every later decision. To quantify this noise floor we re-ran LA-Quantile on the test episodes with its planning targets rounded to single precision, a relative change below 10^{-7}. The plan changed on at least one day in each of the 1,300 test episodes of the thirteen configurations, and episode costs moved by 3.5% (standard deviation) in the base case, between -9.7% and +11.2%, and by 1.6–5.0% across configurations. Averaged over 100 episodes these changes largely cancel (mean change at most 0.48% in absolute value, standard error at most 0.50%). Paired comparisons remain valid, because this variation is part of the episode-level noise they account for, but differences of well below one percent between two lookahead policies should not be interpreted as a systematic effect. The rule-based policies, including UL-DFL, do not solve an optimisation model for their orders and are not affected.

6.9 Verification of the structural results

Table 12 reports numerical checks of the results in Section 3.3 (script `verify_theory.py` in the replication package). Theorem 1(ii) is checked by solving the recourse a second time with FIFO issuing and greedy service imposed through binary indicator constraints: if the theorem were false, the constrained value would exceed V_{LP} in some instance. The bound of Proposition 2(b) is attained in 71% of the random instances, so it cannot be tightened in general. The physical FIFO cost equals V_{LP} in 63% of the random instances; the remaining gap is the value of discarding stock early, which a store does not do in practice. The original submission's claim that age-increasing holding costs alone make every optimal recourse solution FIFO is contradicted by the example given after Theorem 1.

Table 12: Numerical verification of the structural results.

Result	Check	Outcome
Theorem 1(i)	60 lookahead MILPs ($m \in \{2,3,4\}$, 1 or 5 scenarios) with demand from zero to four times fleet capacity	all feasible, all solved to proven gap
Theorem 1(ii)	3,000 random recourse instances ($m \in \{2,3,4\}$, $H \in \{2,\dots,5\}$, random costs); FIFO- and greedy-constrained MILP value vs. V_{LP}	max. absolute difference 10^{-6}
Theorem 1(iii)	$V_{\mathrm{LP}} \le C_{\mathrm{FIFO}}$ in the same instances	holds in 3,000/3,000; equality in 63%
Proposition 2(a)	dual bounds $[-p_u, p_s]$; subgradient inequality in 9,000 random directions	3,000/3,000; 9,000/9,000
Proposition 2(b)	autograd derivative of C_{FIFO} within (7)	3,000/3,000; bound attained in 71%
Proposition 3	$m = 1$, negative-binomial demand (mean 15), SGD on the decision-focused loss	learned target 20.03; empirical 0.8-quantile 20.0; $F(\tilde{d}) = 0.809$ vs. $p_u/(p_u + p_s) = 0.8$
Coverage layer	40 lookahead MILPs; deliveries implied by the visit pattern vs. MILP deliveries	98.7% of delivered volume reproduced (mean relative L_1 deviation 1.3%)

7 Discussion and conclusion

7.1 Managerial implications

The study asked how contextual forecasts should be turned into replenishment decisions when perishable stock ages and deliveries share vehicles. Five implications follow for planners of fresh-food distribution.

1. **Start from a stock-age-based replenishment rule and benchmark against it before integrating.** Daily replenishment at a high service quantile, with the orders routed optimally, was cheaper than every integrated inventory-routing policy in all settings but the one with expensive routing, by 0.9–9.3%, across three network layouts, shelf lives of two to four days, lower and higher lost-sales penalties, tighter and looser fleets, more variable demand, partial LIFO issuing and observed demand of a fresh-food retailer; the margin was smallest when lost sales were cheap, so that high service is worth less. In these settings the travel saved by consolidating deliveries is smaller than the extra holding, waste and lost sales that consolidation causes. This extends the finding of Haijema and Minner [23] and Crama et al. [14] that

well-tuned simple rules are hard to beat to a setting with shared routes and contextual forecasts.

2. **Learn the order-up-to level, not only the forecast.** Training the order-up-to level of the deliver-up-to-level rule on the realised cost of its decisions reduced cost in every setting, significantly in eleven of thirteen, and in the base case it reached the same fill rate with a fifth less waste, which no re-tuning of the rule's service level and trigger achieves; a rule tuned with hindsight matched its cost only by wasting 41% more. On the real data the gain came mainly from fewer weekend stock-outs. The gain was smallest, and not significant, where the rule already wasted little (four-day shelf life) or where demand was too variable for the context to anticipate it, so the learning effort pays most for short-lived products with predictable demand patterns. It also needs the rule: the same network trained to order directly was more expensive than UL-DFL in twelve settings and than the plain rule in eleven. The policy is operated exactly like the rule: it remains a rule with a visit trigger and an order-up-to level that planners can inspect. The learned corrections translate into practice that does not require machine learning: shift safety stock from the days after the weekly peak to the days before it, and, where promotions widen the forecast spread, lower the order-up-to level on promotion days.

3. **Integration pays when routing is expensive, and then uncertainty must be modelled explicitly.** With three times higher travel cost, the scenario-based lookahead of Jia et al. [32] was the cheapest policy and every uncertainty-aware lookahead beat daily base-stock. Deterministic lookahead with mean forecasts was the worst policy in every setting.

4. **If a deterministic plan is used, its safety stock should depend on the planning day.** Decision-focused learning on top of the lookahead learned to keep a high service quantile only for the day that is actually executed and to plan later days near the 0.8 quantile. A lookahead with these two quantile levels, tuned on validation data, removes about half of the waste of the single-quantile plan in the base case and was at least as cheap as the learned targets in all thirteen settings.

5. **Look at store-level service, not only at averages.** Integrated deterministic plans concentrated stock-outs at individual stores, whereas the inventory rules served all stores alike. Retailers with per-store service targets should constrain or monitor this effect.

7.2 Limitations and further research

The synthetic demand is generated by a documented contextual model; the real-data case uses observed sales of one retailer, from which latent demand had to be reconstructed under stock-outs and rescaled to the network's capacities, and from which products whose level shifted strongly were excluded. Exact routing by enumeration limits the network to about a dozen stores; larger networks require heuristic routing, which would weaken the clean separation between replenishment and routing quality used here. Lead time is zero and each store sells one product; positive lead times would increase the value of stock-age information [9], and multiple products would couple stores through shared vehicle space. The lookahead policies relax routing integrality beyond the implemented day, which favours visiting stores more often than a fully integer lookahead would. UL-DFL learns the order-up-to level but keeps the rule's visit trigger and sees fleet capacity only as a proportional scaling during training; learning the trigger as well, or training against the exact routing cost, could extend its advantage to settings with expensive routing, where it was not the best policy. LA-DFL was trained on the decision states visited by LA-Quantile in one round; iterating between training and data collection, or training the scenario generator of ScenPO-SS decision-focusedly, are natural extensions. Finally, deep reinforcement learning has shown promise for single-location perishable inventory [15, 44]; extending it to the routing-coupled action space studied here is an open problem.

7.3 Conclusion

Replenishing perishables on shared routes is a perishable inventory problem first and a routing problem second. Under a common closed-loop simulator with exact optimisation, stock-age-based replenishment rules were the strongest policies without learning unless travel was expensive, in which case scenario-based stochastic lookahead was best. Decision-focused learning made the best rule cheaper in every setting, significantly in eleven of thirteen, by learning from realised cost when to deviate from a fixed service quantile; on top of a deterministic lookahead it revealed why such plans mis-stock, a lesson that a day-specific quantile level implements without learning. The structural results, the experimental protocol and the code are available for testing further policies on the same benchmark.

Data and code availability

All code, configuration files, random seeds and result files needed to reproduce every table and figure are provided in the replication package (`pirp/`, `experiments/`, `results_v2/`). The synthetic demand data are generated by the package. The real-data case uses the public FreshRetailNet-50K dataset [45] (CC BY 4.0), which the package downloads and processes; no proprietary data are used.

References

[1] Aldair Alvarez, Jean-François Cordeau, Raf Jans, Pedro Munari, and Reinaldo Morabito. Formulations, branch-and-cut and a hybrid heuristic algorithm for an inventory routing problem with perishable products. *European Journal of Operational Research*, 283(2):511–529, 2020. doi: 10.1016/j.ejor.2019.11.015. URL https://doi.org/10.1016/j.ejor.2019.11.015.

[2] Matias Alvo, Daniel Russo, and Yash Kanoria. Neural inventory control in networks via hindsight differentiable policy optimization. arXiv preprint arXiv:2306.11246, 2023.

[3] Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 136–145. PMLR, 2017. URL http://proceedings.mlr.press/v70/amos17a.html.

[4] Monique Bakker, Jan Riezebos, and Ruud H. Teunter. Review of inventory systems with deterioration since 2001. *European Journal of Operational Research*, 221(2):275–284, 2012. doi: 10.1016/j.ejor.2012.03.004.

[5] Gah-Yi Ban and Cynthia Rudin. The big data newsvendor: Practical insights from machine learning. *Operations Research*, 67(1):90–108, 2019. doi: 10.1287/opre.2018.1757. URL https://doi.org/10.1287/opre.2018.1757.

[6] Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers. In *Advances in Neural Information Processing Systems*, volume 33, pages 9008–9019, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/66808e327dc79d135ba18e051673d906-Abstract.html.

[7] Dimitris Bertsimas and Aurélie Thiele. A robust optimization approach to inventory theory. *Operations Research*, 54(1):150–168, 2006. doi: 10.1287/opre.1050.0238.

[8] Robert N. Boute, Joren Gijsbrechts, Willem van Jaarsveld, and Nathalie Vanvuchelen. Deep reinforcement learning for inventory control: A roadmap. *European Journal of*

Operational Research, 298(2):401–412, 2022. doi: 10.1016/j.ejor.2021.07.016. URL https://doi.org/10.1016/j.ejor.2021.07.016.

[9] Rob A. C. M. Broekmeulen and Karel H. van Donselaar. A heuristic to manage perishable inventory with batch ordering, positive lead-times, and time-varying demand. *Computers & Operations Research*, 36(11):3013–3018, 2009. doi: 10.1016/j.cor.2009.01.017.

[10] Jinzhi Bu, Xiting Gong, and Xiuli Chao. Asymptotic optimality of base-stock policies for perishable inventory systems. *Management Science*, 69(2):846–864, 2023. doi: 10.1287/mnsc.2022.4400.

[11] Leandro C. Coelho and Gilbert Laporte. Optimal joint replenishment, delivery and inventory management policies for perishable products. *Computers & Operations Research*, 47:42–52, 2014. doi: 10.1016/j.cor.2014.01.013. URL https://doi.org/10.1016/j.cor.2014.01.013.

[12] Leandro C. Coelho, Jean-François Cordeau, and Gilbert Laporte. Heuristics for dynamic and stochastic inventory-routing. *Computers & Operations Research*, 52:55–67, 2014. doi: 10.1016/j.cor.2014.07.001.

[13] Leandro C. Coelho, Jean-François Cordeau, and Gilbert Laporte. Thirty years of inventory routing. *Transportation Science*, 48(1):1–19, 2014. doi: 10.1287/trsc.2013.0472. URL https://doi.org/10.1287/trsc.2013.0472.

[14] Yves Crama, Mahmood Rezaei, Martin W. P. Savelsbergh, and Tom van Woensel. Stochastic inventory routing for perishable products. *Transportation Science*, 52(3):526–546, 2018. doi: 10.1287/trsc.2017.0799. URL https://doi.org/10.1287/trsc.2017.0799.

[15] Bram J. De Moor, Joren Gijsbrechts, and Robert N. Boute. Reward shaping to improve the performance of deep reinforcement learning in perishable inventory management. *European Journal of Operational Research*, 301(2):535–545, 2022. doi: 10.1016/j.ejor.2021.10.045.

[16] Priya L. Donti, Brandon Amos, and J. Zico Kolter. Task-based end-to-end model learning in stochastic optimization. In *Advances in Neural Information Processing Systems*, volume 30, pages 5484–5494, 2017. doi: 10.48550/arXiv.1703.04529. URL https://doi.org/10.48550/arXiv.1703.04529.

[17] Adam N. Elmachtoub and Paul Grigas. Smart "predict, then optimize". *Management Science*, 68(1):9–26, 2022. doi: 10.1287/mnsc.2020.3922. URL https://doi.org/10.1287/mnsc.2020.3922.

[18] Adam N. Elmachtoub, Henry Lam, Haofeng Zhang, and Yunfan Zhao. Estimate-then-optimize versus integrated-estimation-optimization versus sample average approximation: A stochastic dominance perspective. arXiv preprint arXiv:2304.06833, 2023.

[19] Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022. doi: 10.1287/msom.2021.1064.

[20] Toni Greif, Louis Bouvier, Christoph M. Flath, Axel Parmentier, Sonja U. K. Rohmer, and Thibaut Vidal. Combinatorial optimization and machine learning for dynamic inventory routing. arXiv preprint arXiv:2402.04463, 2024.

[21] René Haijema. Optimal ordering, issuance and disposal policies for inventory management of perishable products. *International Journal of Production Economics*, 157:158–169, 2014. doi: 10.1016/j.ijpe.2014.06.014. URL https://doi.org/10.1016/j.ijpe.2014.06.014.

[22] René Haijema and Stefan Minner. Stock-level dependent ordering of perishables: A comparison of hybrid base-stock and constant order policies. *International Journal of Production Economics*, 181:215–225, 2016. doi: 10.1016/j.ijpe.2015.10.013.

[23] René Haijema and Stefan Minner. Improved ordering of perishables: The value of stock-age information. *International Journal of Production Economics*, 209:316–324, 2019. doi: 10.1016/j.ijpe.2018.03.008.

[24] Zhichao He, Ya Liu, and Kun Liu. Robust optimization approaches for the perishable product inventory routing problem with demand uncertainty. *Journal of Industrial and Management Optimization*, 20(8):2740–2769, 2024. doi: 10.3934/jimo.2024024. URL https://doi.org/10.3934/jimo.2024024.

[25] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962. doi: 10.1137/0110015.

[26] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. doi: 10.2307/4615733. URL https://doi.org/10.2307/4615733.

[27] Yichun Hu, Nathan Kallus, and Xiaojie Mao. Fast rates for contextual linear optimization. *Management Science*, 68(6):4236–4245, 2022. doi: 10.1287/mnsc.2022.4383.

[28] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[29] Jakob Huber, Sebastian Müller, Moritz Fleischmann, and Heiner Stuckenschmidt. A data-driven newsvendor problem: From data to decision. *European Journal of Operational Research*, 278(3):904–915, 2019. doi: 10.1016/j.ejor.2019.04.043.

[30] MD Shafikul Islam Sohan and Azmine Toushik Wasi. Optimizing inventory routing: A decision-focused learning approach using neural networks. In *New in Machine Learning Workshop, NeurIPS 2023 (Extended Abstracts)*, 2023. URL https://openreview.net/forum?id=r0fzjB8f7f. Workshop extended abstract; arXiv:2311.00983.

[31] Larissa Janssen, Thorsten Claus, and Jürgen Sauer. Literature review of deteriorating inventory models by key topics from 2012 to 2015. *International Journal of Production Economics*, 182:86–112, 2016. doi: 10.1016/j.ijpe.2016.08.019.

[32] Menglei Jia, Albert H. Schrotenboer, and Feng Chen. Scenario predict-then-optimize for data-driven online inventory routing. *Transportation Science*, 59(5):1032–1056, 2025. doi: 10.1287/trsc.2024.0613. URL https://doi.org/10.1287/trsc.2024.0613.

[33] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[34] Sebastian Malicki and Stefan Minner. Cyclic inventory routing with dynamic safety stocks under recurring non-stationary interdependent demands. *Computers & Operations Research*, 131:105247, 2021. doi: 10.1016/j.cor.2021.105247.

[35] Jayanta Mandi, James Kotary, Senne Berden, Maxime Mulamba, Víctor Bucarey, Tias Guns, and Ferdinando Fioretto. Decision-focused learning: Foundations, state of the art, benchmark

and future opportunities. *Journal of Artificial Intelligence Research*, 80:1623–1701, 2024. doi: 10.1613/jair.1.15320. URL https://doi.org/10.1613/jair.1.15320.

[36] Maxime Mulamba, Jayanta Mandi, Michelangelo Diligenti, Michele Lombardi, Victor Bucarey, and Tias Guns. Contrastive losses and solution caching for predict-and-optimize. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)*, pages 2833–2840, 2021. doi: 10.24963/ijcai.2021/390.

[37] Steven Nahmias. Perishable inventory theory: A review. *Operations Research*, 30(4):680–708, 1982. doi: 10.1287/opre.30.4.680. URL https://doi.org/10.1287/opre.30.4.680.

[38] Warren B. Powell. A unified framework for stochastic optimization. *European Journal of Operational Research*, 275(3):795–821, 2019. doi: 10.1016/j.ejor.2018.07.014.

[39] Utsav Sadana, Abhilash Chenreddy, Erick Delage, Alexandre Forel, Emma Frejinger, and Thibaut Vidal. A survey of contextual optimization methods for decision-making under uncertainty. *European Journal of Operational Research*, 320(2):271–289, 2025. doi: 10.1016/j.ejor.2024.03.020. URL https://doi.org/10.1016/j.ejor.2024.03.020.

[40] Marius M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987. doi: 10.1287/opre.35.2.254. URL https://doi.org/10.1287/opre.35.2.254.

[41] Oğuz Solyalı, Jean-François Cordeau, and Gilbert Laporte. Robust inventory routing under demand uncertainty. *Transportation Science*, 46(3):327–340, 2012. doi: 10.1287/trsc.1110.0387.

[42] Mehmet Soysal, Jacqueline M. Bloemhof-Ruwaard, René Haijema, and Jack G. A. J. van der Vorst. Modeling an inventory routing problem for perishable products with environmental considerations and demand uncertainty. *International Journal of Production Economics*, 164:118–133, 2015. doi: 10.1016/j.ijpe.2015.03.008. URL https://doi.org/10.1016/j.ijpe.2015.03.008.

[43] Bo Tang and Elias B. Khalil. PyEPO: A PyTorch-based end-to-end predict-then-optimize library for linear and integer programming. *Mathematical Programming Computation*, 16(3):297–335, 2024. doi: 10.1007/s12532-024-00255-x. URL https://doi.org/10.1007/s12532-024-00255-x.

[44] Tarkan Temizöz, Christina Imdahl, Remco Dijkman, Douniel Lamghari-Idrissi, and Willem van Jaarsveld. Deep controlled learning for inventory control. *European Journal of Operational Research*, 324(1):104–117, 2025. doi: 10.1016/j.ejor.2025.01.026.

[45] Yangyang Wang, Jiawei Gu, Li Long, Xin Li, Li Shen, Zhouyu Fu, Xiangjun Zhou, and Xu Jiang. FreshRetailNet-50K: A stockout-annotated censored demand dataset for latent demand recovery and forecasting in fresh retail. arXiv preprint arXiv:2505.16319, 2025. Dataset: https://huggingface.co/datasets/Dingdong-Inc/FreshRetailNet-50K, CC BY 4.0.

A Proofs

Proof of Proposition 1. Fix v and k and let $R^\star = \{i : z_{ivk} = 1\}$. Constraint (3) gives $\sum_R y_{Rvk} = 1$ and $\sum_{R \ni i} y_{Rvk} = z_{ivk}$. For $i \in R^\star$ the second equality equals the first, so every R with $y_{Rvk} > 0$ contains i; for $i \notin R^\star$ the sum is zero, so no such R contains i. Hence $y_{Rvk} > 0$ implies $R = R^\star$, and $y_{R^\star vk} = 1$. The routing cost is $\mathrm{TSP}(R^\star)$, vehicle loads are at most Q by (4), and each store

is on at most one vehicle. Every feasible lookahead plan (stores per vehicle, loads, inventory flows) has exactly one representation of this form, which proves exactness. □

Proof of Theorem 1. (i) Setting $z = 0$, $y_{\emptyset vk} = 1$, $q = 0$, $\zeta = 0$ and serving demand from available stock in any order is feasible: day-0 stock satisfies the shelf constraint by assumption and stock only decreases afterwards. All cost terms (travel, holding of carried units, waste, lost sales) are non-negative, so the objective is bounded below by zero; with finitely many assignments and a bounded linear programme for each, an optimum exists.

(ii) Take an optimal recourse solution and view inventory as fungible flows of units per cohort. *Greedy service.* If on day k demand is lost while $\varepsilon > 0$ units are left on the shelf, sell ε of them. Those units originally had one of four fates: sold on a later day (that later sale is now lost; the cost change is $-p_u\varepsilon + p_u\varepsilon - h\varepsilon \cdot (\text{nights saved}) \le 0$), expired or disposed (saving $p_s\varepsilon$ and holding), or carried beyond the horizon (saving holding). The cost does not increase. *FIFO.* If on day k a younger cohort r' is sold while ε units of an older cohort $r < r'$ are left, sell ε older units instead and keep ε younger ones. The kept younger units replicate the original path of the older units: they are sellable on every day the older units were (their remaining life is longer); where the older units expired at the end of day $k + r - 1$, the younger units are disposed on that day at the same cost p_s and with the same number of holding nights; disposal and carrying beyond the horizon are replicated identically. The number of units carried each night is unchanged, so the cost is unchanged. Each exchange reduces $\sum_k \sum_r r\, w_{irk}$ and the feasible set is a polytope, so a finite sequence of exchanges yields an optimal solution with FIFO issuing and greedy service.

(iii) q enters the recourse LP only through the right-hand side of $S_{imk} = q_k$, so V_{LP} is the value function of an LP in its right-hand side, which is convex and piecewise linear. The physical FIFO trajectory with $\zeta = 0$ is feasible for the recourse LP and has cost C_{FIFO}; hence $V_{\mathrm{LP}} \le C_{\mathrm{FIFO}}$. By (ii) there is an optimal solution with FIFO issuing and greedy service; if it uses no disposal it coincides with the physical trajectory and the two values are equal. □

Proof of Proposition 2. (a) By LP duality, the optimal dual of the constraint $S_{imk} = q_k$ is a subgradient of the value function with respect to q_k. Adding ε to q_k and disposing of these units immediately is feasible and costs $p_s\varepsilon$ (holding $h\varepsilon$ through S_{imk} plus $(p_s - h)\varepsilon$ through ζ), so $V_{\mathrm{LP}}(q + \varepsilon e_k) \le V_{\mathrm{LP}}(q) + p_s\varepsilon$ and $\pi_k \le p_s$. Removing $\varepsilon \le q_k$ units turns at most ε sales into lost sales and cannot increase holding or waste, so $V_{\mathrm{LP}}(q - \varepsilon e_k) \le V_{\mathrm{LP}}(q) + p_u\varepsilon$ and $\pi_k \ge -p_u$.

(b) C_{FIFO} is a composition of finitely many min, max and affine maps of q, hence piecewise linear and Lipschitz. Compare the FIFO trajectories with deliveries q and $q + \varepsilon e_k$. On day k the extra units are the freshest and are sold last, so sales of older stock are unchanged. On later days the extra units are older than subsequent deliveries and may displace them, but stock that was older than the extra units is never affected. Therefore, relative to the original trajectory, lost sales can only fall, waste and carried stock can only rise, and the total extra stock E_ℓ carried on night ℓ satisfies $0 \le E_\ell \le \varepsilon$. Each extra unit leaves the system by preventing a lost sale ($-p_u$), by expiring on some day $\tau \le H - 1$ ($+p_s$ after at most $\tau - k \le H - 1 - k$ nights), or by remaining at the end ($H - k$ nights). The cost change per unit is therefore at least $-p_u$ and at most $\max\{p_s + (H - 1 - k)h, (H - k)h\} = p_s + (H - 1 - k)h$ when $h \le p_s$. Dividing by ε gives (7) for both one-sided derivatives. □

Proof of Proposition 3. With the visit pattern fixed and no capacity binding, the coverage map gives $q_k = \sum_{j \in W_k} \tilde{d}_j - U_k(\tilde{d})$, where U_k is the planned usable stock that serves the window. A unit increase of $\tilde{d}_j$, $j \in W_k$, increases q_k by one unless the planned stock already covers day j, in which case the target of day j does not affect any delivery; days outside every window affect no delivery. The chain rule then gives (8), and setting it to zero is the first-order condition of $\min_q \mathbb{E}[C_{\mathrm{FIFO}}(q; D) \mid \xi]$ over the deliveries of the fixed pattern. With $m = 1$ nothing is carried, $W_k = \{k\}$, $q_k = \tilde{d}_k$ and $C_{\mathrm{FIFO}} = \sum_k p_u(D_k - q_k)^+ + p_s(q_k - D_k)^+$, so $\mathbb{E}[\partial C/\partial q_k \mid \xi] = -p_u\mathbb{P}(D_k >$

$q_k \mid \xi) + p_s \mathbb{P}(D_k < q_k \mid \xi)$, which vanishes when $F_{D_k|\xi}(\tilde{d}_k) = p_u/(p_u + p_s)$ (for continuous F; for discrete demand, at the smallest $\tilde{d}_k$ with $F \geq p_u/(p_u + p_s)$). $\square$